

Hydra Developer Guide

Table of Contents

Preface	1
1. Part I: Architecture and Development Setup	2
2. Architecture and Development Setup	3
2.1. Overview	3
2.1.1. Supported Language Variants	3
2.1.2. Key Benefits of the Pixi Setup	3
2.2. Project Architecture	3
2.2.1. Core Configuration Files	3
2.2.2. Package Hierarchy	4
2.2.3. Package Structure	5
2.2.4. Integration with Existing Build Systems	6
2.2.5. Command-Line Interface	6
3. Pixi Setup and Installation	7
3.1. Prerequisites	7
3.1.1. System Requirements	7
3.1.2. Platform Support	7
3.2. Quick Start	7
3.2.1. Basic Setup	7
3.2.2. Environment-Specific Setup	7
3.2.3. Verification	8
3.3. Troubleshooting	8
3.3.1. Common Installation Issues	8
3.3.2. Common Build Issues	9
3.3.3. Environment Issues	11
3.3.4. Performance Issues	12
3.3.5. Getting Help	12
3.3.6. Diagnostic Commands	12
3.3.7. Clean Installation	13
4. Build System	14
4.1. Available Environments	14
4.2. Build Tasks	14
4.2.1. General Build Tasks	14
4.2.2. Using Environments	14
4.3. Language-Specific Development Tasks	15
4.3.1. Java Development Tasks	15
4.3.2. Python Development Tasks	15
4.3.3. Scala Development Tasks	15
4.4. Development Workflow	16

4.4.1. Making Changes	16
4.4.2. Testing Changes	16
4.5. Advanced Usage	16
4.5.1. Custom Build Options	16
4.5.2. Recipe Format Details	16
4.5.3. Publishing Packages	17
4.6. Contributing	17
4.6.1. Code Changes	17
4.6.2. Testing Contributions	17
4.6.3. Pixi Configuration Guidelines	18
5. Part II: Haskell Development	19
6. Haskell Development Setup Guide	20
6.1. Prerequisites	20
6.1.1. System Dependencies	20
6.1.2. GHC and Stack Installation via ghcup	20
6.2. Project Structure	20
6.3. Getting Started	21
6.3.1. Using Pixi Tasks	21
6.4. Common Workflows	21
6.4.1. Development with GHCi	21
6.4.2. Running Tests	22
6.4.3. Building and Installing	22
6.5. Troubleshooting	22
6.5.1. Common Issues	22
6.5.2. GHCi Package Loading Issues	22
6.5.3. Build Failures	23
6.6. IDE Integration	23
6.6.1. VS Code with Haskell Language Server	23
6.6.2. Vim/Neovim	23
6.7. GraphSON Generation Tasks	23
6.8. Package Dependencies	23
6.9. Useful Commands	24
6.10. Key Benefits of This Setup	24
6.11. Quick Reference	24
7. Pixi Haskell Development Tasks Reference	25
7.1. Prerequisites	25
7.2. Interactive Development	25
7.2.1. GHCi Sessions	25
7.2.2. Available Modules in GHCi	25
7.3. Building and Testing	26
7.3.1. Build Tasks	26

7.3.2. Testing Tasks	26
7.3.3. Setup and Maintenance	26
7.4. Code Exploration	27
7.4.1. Module Discovery	27
7.5. GraphSON Generation (hydra-ext)	27
7.5.1. Check Existing Data	27
7.5.2. Generate GraphSON Data	27
7.6. Unified Development Tasks	28
8. Unified Haskell tasks	29
8.1. Advanced Development Tasks	29
8.1.1. Alternative GHCi Tasks	29
8.1.2. Code Generation Tasks	29
8.2. Troubleshooting	29
8.2.1. Previous Issues (Now Resolved with ghcup)	30
8.2.2. Current Working Solutions	30
8.2.3. Environment Verification	31
8.3. Standard Workflow (All Tasks Working)	31
8.3.1. Primary ghcup-based Tasks	31
8.3.2. Direct Stack Commands (Now Recommended)	31
8.4. Files Created by Setup	32
8.5. Current Status Summary	32
8.5.1. [] All Tasks Working (25 tasks)	32
8.5.2. Key Success Factors	32
8.5.3. Recommendation	32
8.5.4. Cross-Platform Development	33
8.6. Task Categories Summary	33
8.6.1. [+] Setup Tasks (5 tasks)	33
8.6.2. [+] Build Tasks (8 tasks)	33
8.6.3. [+] Test Tasks (6 tasks)	33
8.6.4. [+] GHCi Tasks (8 tasks)	34
8.6.5. [+] Data and GraphSON Tasks (8 tasks)	34
8.6.6. [+] Exploration Tasks (2 tasks)	34
8.6.7. [+] Cleanup Tasks (6 tasks)	34
8.7. Related Documentation	34
9. Pixi Task Status and Repair Guide	36
9.1. Failing Tasks	36
9.1.1. Main Build Tasks	36
9.1.2. Haskell Environment Tasks (Build-related)	36
9.1.3. GraphSON Generation Tasks	36
9.2. Outstanding Issues Requiring Attention	37
9.2.1. Java Build Issues	37

9.2.2. Haskell Linking Issues	37
9.2.3. Multi-package Dependency Issues	37
9.2.4. Scala Build Issues	38
9.2.5. GraphSON Generation	38
9.3. Working Environments	38
9.3.1. Java Environment	38
9.3.2. Python Environment	38
9.3.3. Haskell Environment	38
9.3.4. Scala Environment	39
9.4. Recommendations	39
9.4.1. Immediate Actions	39
9.4.2. Medium-term Solutions	39
9.4.3. Long-term Improvements	39
9.5. Usage Recommendations	39
9.5.1. For Development	39
9.5.2. For CI/CD	39
9.6. Documentation Updates Applied	40
9.7. Next Steps	40
10. Part III: Dependency Tiers and Code Organization	41
11. Dependency Tiers and Code Organization	42
11.1. Tier-0: Kernel Modules	42
11.2. Tier-0: DSLs	42
11.3. Tier-1: Kernel Modules	42
11.4. Tier-1: Primitive Functions	42
11.5. Tier-1: DSLs	42
11.6. Tier-2: Kernel Modules	43
11.7. Tier-2: Primitive Functions	43
11.8. Tier-3: Kernel Sources	43
11.9. Tier-4: Non-Kernel Code	43
11.10. Tier-5: Third-Party Code	43
11.11. Dependency Rules and Guidelines	43
11.11.1. Enforcement Rules	43
11.11.2. Design Guidelines	44
11.11.3. Common Patterns	44
11.12. Working with Dependency Tiers	45
11.12.1. Development Workflow	45
11.12.2. Debugging Dependency Issues	45
11.12.3. Tier Verification	45
11.13. Advanced Tier Concepts	46
11.13.1. Cross-Language Tiers	46
11.13.2. Tier Migration	46

11.13.3. Custom Tier Extensions	46
11.14. Related Documentation	46
12. Hydra Dependency Tiers: Directory Structure and File Organization	47
12.1. Overview	47
12.2. Tier-0: Foundation Tier	48
12.2.1. Directory Structure	49
12.2.2. Key Characteristics	49
12.3. Tier-1: Kernel Modules with Primitives	50
12.3.1. Directory Structure	50
12.3.2. Key Characteristics	50
12.4. Tier-2: Advanced Kernel Modules	50
12.4.1. Directory Structure	51
12.4.2. Key Characteristics	51
12.5. Tier-3: Kernel Sources and Code Generation	52
12.5.1. Directory Structure	52
12.5.2. Key Characteristics	53
12.6. Tier-4: Non-Kernel Code and Language Coders	53
12.6.1. Directory Structure	53
12.6.2. Key Characteristics	54
12.7. Tier-5: Third-Party Code and User Extensions	54
12.7.1. Directory Structure	54
12.7.2. Key Characteristics	55
12.8. Build and Generation Process	55
12.8.1. Code Generation Flow	55
12.8.2. Key Generation Commands	55
12.9. Dependencies and Import Rules	55
12.9.1. Allowed Dependencies	55
12.9.2. Forbidden Dependencies	56
12.10. Maintenance Guidelines	56
12.10.1. Source of Truth Priority	56
12.10.2. Adding New Functionality	56
12.10.3. Evolution Considerations	56
12.11. Related Documentation	56
13. Part IV: Code Generation	57
14. Code Generation and Generated Code	58
14.1. Overview	58
14.1.1. Code Generation Pipeline	58
14.1.2. Generated vs. Hand-Written Code	58
14.2. Working with Generated Code	58
14.2.1. Interactive Code Generation	58
14.2.2. Automated Code Generation	59

14.2.3. Verification After Generation	59
14.3. Code Generation Architecture	59
14.3.1. Source Definition Structure	59
14.3.2. Module Registration	60
14.4. Language-Specific Generation	60
14.4.1. Haskell Code Generation	60
14.4.2. Java Code Generation	61
14.4.3. Python Code Generation (Future)	61
14.4.4. Scala Code Generation (Future)	61
14.5. Customizing Code Generation	61
14.5.1. Adding New Language Support	61
14.5.2. Customizing Existing Generators	62
14.6. Generated Code Organization	62
14.6.1. Directory Structure	62
14.6.2. Module Naming Conventions	62
14.6.3. Version Synchronization	63
14.7. Development Workflow	63
14.7.1. Iterative Development	63
14.7.2. Debugging Generated Code	63
14.7.3. Testing Generated Code	64
14.8. Code Generation Best Practices	64
14.8.1. DSL Design Guidelines	64
14.8.2. Generation Quality	64
14.8.3. Version Control	64
14.9. Advanced Topics	64
14.9.1. Custom Code Templates	64
14.9.2. Conditional Generation	65
14.9.3. Cross-Language Testing	65
14.10. Troubleshooting	65
14.10.1. Common Issues	65
14.10.2. Diagnostic Commands	66
14.11. Related Documentation	66
15. Part V: Documentation System	67
16. Documentation System	68
16.1. Overview	68
16.1.1. Key Features	68
16.1.2. Documentation Architecture	68
16.2. Build System	68
16.2.1. Pixi Tasks for Documentation	68
16.2.2. Build Dependencies	69
16.2.3. Build Process Flow	69

16.3. Authoring Documentation	70
16.3.1. File Organization	70
16.3.2. AsciiDoc Conventions	70
16.3.3. PlantUML Diagrams	72
16.4. GitHub Pages Integration	73
16.4.1. Automated Deployment	73
16.4.2. Deployment Process	74
16.4.3. Custom Domain Support	74
16.5. Development Workflow	74
16.5.1. Daily Documentation Work	74
16.5.2. Before Committing Changes	74
16.5.3. Release Documentation	75
16.6. Maintenance and Best Practices	75
16.6.1. Content Guidelines	75
16.6.2. Diagram Guidelines	75
16.6.3. Quality Assurance	75
16.6.4. Performance Considerations	75
16.7. Troubleshooting	76
16.7.1. Common Issues	76
16.7.2. Debugging Techniques	76
16.8. Future Enhancements	76
16.8.1. Planned Improvements	77
16.8.2. Extension Opportunities	77
16.9. Contributing to Documentation	77
16.9.1. Getting Started	77
16.9.2. Contribution Process	77
16.9.3. Review Criteria	77
17. Part VI: Release Management	79
18. Release Management	80
18.1. Release Overview	80
18.1.1. Release Components	80
18.1.2. Version Strategy	80
18.2. Pre-Release Process	80
18.2.1. Version Planning	80
18.2.2. Pre-Release Readiness Checks	80
18.3. Hydra-Haskell Release Process	81
18.3.1. Version Preparation	82
18.3.2. Release to Hackage	82
18.3.3. Tag the Release	83
18.4. Hydra-Java Release Process	83
18.4.1. Pre-Release Steps	83

18.4.2. Release to Maven Central	84
18.5. Multi-Component Release Coordination	85
18.5.1. Version Synchronization	85
18.5.2. Testing Cross-Implementation Compatibility	85
18.6. Hotfix Release Process	86
18.6.1. 1. Create Hotfix Branch	86
18.6.2. 2. Implement and Test Fix	86
18.6.3. 3. Update Version (Patch Only)	86
18.6.4. 4. Merge and Release	86
18.7. Post-Release Tasks	86
18.7.1. 1. Update GitHub Release	86
18.7.2. 2. Update Documentation	87
18.7.3. 3. Notify Community	87
18.7.4. 4. Monitor for Issues	87
18.8. Release Quality Checklist	87
18.9. Emergency Procedures	87
18.9.1. Reverting a Release	88
18.9.2. Hotfix Communication	88
18.10. Automation and Tools	88
18.10.1. Release Scripts	88
18.10.2. CI/CD Integration	89
18.11. Related Documentation	89
19. Appendices	90
Appendix A: Development Resources	90
Appendix B: Common Development Tasks	90
19.B.1. Setting Up Development Environment	90
Appendix C: Complete Pixi Tasks Reference	90
19.C.1. General Build and Setup Tasks	90
19.C.2. Java Development Tasks	91
19.C.3. Python Development Tasks	91
19.C.4. Scala Development Tasks	91
19.C.5. Haskell Development Tasks	91
19.C.6. Development Workflow Recommendations	93
19.C.7. Debugging Build Issues	93
19.C.8. Contributing Guidelines	93
Colophon	94

Preface

This guide is for developers of Hydra itself - those working on the core Hydra kernel, build system, and language implementations. If you're looking to create new Hydra implementations, see the [Implementor Guide](#). If you're using Hydra to create message translators, see the [Operator Guide](#).

For general contribution guidelines, see the [Contributors](#) page.

Chapter 1. Part I: Architecture and Development Setup

Chapter 2. Architecture and Development Setup

This section covers the overall architecture of the Hydra project and how to set up your development environment.

2.1. Overview

The Hydra project is a multi-language functional programming toolkit that has been successfully converted from a traditional build setup to a modern pixi project. This conversion enables unified conda package generation for all language variants using rattler-build.

2.1.1. Supported Language Variants

- **Java** - Core implementation with complete functionality
- **Python** - Python bindings and implementation (in progress)
- **Haskell** - Pure functional implementation
- **Scala** - JVM-based functional implementation
- **Extensions** - Additional functionality supporting both Java and Haskell

2.1.2. Key Benefits of the Pixi Setup

1. **Unified Build System** - Single command to build all variants
2. **Modern Recipe Format** - Uses rattler-build v1 specification with pure YAML
3. **Cross-Platform Scripts** - Nushell provides consistent behavior across platforms
4. **Dependency Management** - Pixi handles all build tool dependencies
5. **Environment Isolation** - Clean, reproducible build environments
6. **Package Distribution** - Easy conda package creation and distribution
7. **Developer Experience** - Simple setup with `pixi install`
8. **CI/CD Ready** - Suitable for automated build pipelines

2.2. Project Architecture

2.2.1. Core Configuration Files

- **pixi.toml** - Main pixi project configuration
 - Defines project metadata and version (0.12.0)
 - Specifies conda-forge as the primary channel
 - Configures multi-platform support (linux-64, osx-64, osx-arm64, win-64)
 - Sets up feature-based environments for different language toolchains

- Defines build tasks for all variants
- **recipes/** directory containing rattler-build v1 recipes with embedded nushell scripts

2.2.2. Package Hierarchy

The project generates several conda packages:

Individual Language Packages

1. **hydra-java** (v0.12.0)

- Built with Gradle via nushell script
- Requires OpenJDK 11+ and nushell
- Generates JAR files and wrapper scripts
- Installs to **\$PREFIX/lib/hydra-java/**

2. **hydra-python** (v0.10.0)

- Built with pip + hatchling via nushell script
- Requires Python 3.12+ and nushell
- Standard Python package installation
- Uses existing **pyproject.toml**

3. **hydra-haskell** (v0.12.0)

- Built with Stack via nushell script
- Requires GHC 9.4+, Stack, and nushell
- Generates libraries and executables
- Installs to **\$PREFIX/lib/hydra-haskell/**

4. **hydra-scala** (v0.1.0)

- Built with SBT via nushell script
- Requires OpenJDK 11+, SBT, and nushell
- Generates JAR files and wrapper scripts
- Installs to **\$PREFIX/lib/hydra-scala/**

5. **hydra-ext** (v0.12.0)

- Hybrid Java/Haskell build via nushell script
- Supports both Gradle and Stack builds
- Contains additional functionality
- Includes data files and extensions

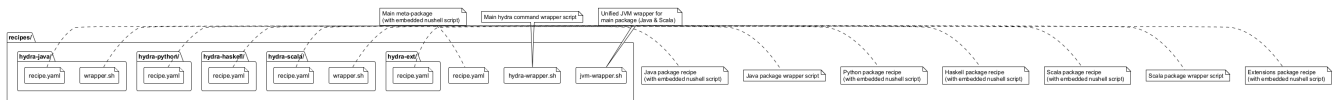
Meta-Package

The main **hydra** package (v0.12.0) is a meta-package that:

- Depends on all individual variant packages
- Provides unified command-line interface
- Includes comprehensive wrapper script (**hydra** command)
- Supports multi-variant usage patterns

2.2.3. Package Structure

Recipe Organization



Modern Recipe Format

Uses rattler-build v1 specification with these improvements:

- **Pure YAML format** - No external bash/batch files
- **Embedded nushell scripts** - Cross-platform build logic
- **Proper Jinja syntax** - `{{{ variable }}}` instead of `{{ variable }}`
- **Updated metadata fields** - `homepage` instead of `home`
- **Enhanced validation** - JSON schema support
- **V1 test format** - `tests:` instead of `test:`

Build Process Details

All recipes use **nushell scripts** embedded directly in the `recipe.yaml` files. This provides:

- Pure YAML format (no separate bash/batch files)
- Cross-platform compatibility through nushell
- Better integration with rattler-build
- Improved error handling and logging
- Separate wrapper script files for better maintainability
- Unified JVM wrapper reduces duplication for main package

Cross-Platform Support

Recipes are configured for multiple platforms:

- Linux x86_64 (**linux-64**)
- macOS Intel (**osx-64**)
- macOS Apple Silicon (**osx-arm64**)
- Windows x86_64 (**win-64**)

2.2.4. Integration with Existing Build Systems

The pixi setup works alongside existing build configurations:

- Preserves existing `build.gradle` files
- Uses existing `pyproject.toml` for Python
- Leverages existing `stack.yaml` for Haskell
- Utilizes existing `build.sbt` for Scala

No changes were made to the core language implementations - the pixi layer provides conda packaging on top of existing build systems via nushell scripts that call the appropriate build tools.

2.2.5. Command-Line Interface

After installation, users can access Hydra through multiple interfaces:

Multi-Variant Wrapper

```
# Main hydra command (multi-variant wrapper)
hydra --help
hydra java --help
hydra python -c "import hydra; print('Hydra Python loaded')"
hydra haskell --version
hydra scala --version
```

Direct Variant Access

```
# Direct variant commands
hydra-java --help
hydra-scala --help
# Plus any generated Haskell executables
```

Language-Specific APIs

Python API

```
import hydra
# Use Hydra Python implementation
```

Haskell Library

```
import Hydra.Core
-- Use Hydra Haskell implementation
```

Chapter 3. Pixi Setup and Installation

This section covers getting started with the Hydra pixi-based development environment, including prerequisites, installation, and troubleshooting common issues.

3.1. Prerequisites

3.1.1. System Requirements

1. **Install pixi:** Follow the installation instructions at <https://pixi.sh>
2. **System Requirements:** Ensure you have sufficient disk space (building all variants requires several GB)
3. **Network Access:** Required for downloading dependencies from conda-forge and other package repositories

3.1.2. Platform Support

The Hydra project supports the following platforms through pixi:

- Linux x86_64 (`linux-64`)
- macOS Intel (`osx-64`)
- macOS Apple Silicon (`osx-arm64`)
- Windows x86_64 (`win-64`)

3.2. Quick Start

3.2.1. Basic Setup

```
# Clone the repository
git clone https://github.com/CategoricalData/hydra
cd hydra

# Install all build dependencies
pixi install

# Validate the setup
pixi run validate

# Build all packages
pixi run build-all
```

3.2.2. Environment-Specific Setup

```
# Install dependencies for specific environments
```

```
pixi install -e java      # Java development environment
pixi install -e python    # Python development environment
pixi install -e haskell   # Haskell development environment
pixi install -e scala     # Scala development environment
pixi install -e all       # Complete multi-language environment
```

3.2.3. Verification

After installation, verify your setup:

```
# Check pixi installation
pixi --version

# List available environments
pixi info

# Check specific environment
pixi list -e java
pixi list -e python
pixi list -e haskell
pixi list -e scala

# Test basic functionality
pixi run validate
```

3.3. Troubleshooting

3.3.1. Common Installation Issues

Pixi Installation Problems

- **Issue:** `pixi` command not found
- **Solution:** Ensure pixi is properly installed and in your PATH. Follow installation instructions at <https://pixi.sh>
- **Issue:** Permission denied errors
- **Solution:** Check file permissions and ensure you have write access to the installation directory

Dependency Resolution Issues

- **Issue:** Failed to solve dependencies
- **Solution:** Check network connectivity and try clearing pixi cache:

```
# Clear pixi cache
pixi clean cache
```

```
# Retry installation
pixi install
```

3.3.2. Common Build Issues

Nushell Script Issues

- **Issue:** Nushell commands fail or are not recognized
- **Symptoms:** Build scripts fail with nushell-related errors
- **Solutions:**
 - Ensure nushell is available: `pixi list | grep nushell`
 - Check nushell syntax in recipe files
 - Verify environment variables are properly referenced (`$env.PREFIX`, `$env.SRC_DIR`)

Java Build Failures

- **Issue:** Java build fails or Gradle errors
- **Symptoms:** `pixi run build-java` or `pixi run -e java gradle-build` fails
- **Solutions:**
 - Ensure OpenJDK 11+ is available in the java environment
 - Check that Gradle can resolve dependencies
 - Verify network connectivity for dependency downloads
 - Check nushell file operations (glob patterns, path operations)

```
# Debug Java environment
pixi run -e java java -version
pixi run -e java gradle --version

# Clean and rebuild
pixi run -e java gradle-clean
pixi run -e java gradle-build
```

Python Build Failures

- **Issue:** Python build fails or pip errors
- **Symptoms:** `pixi run build-python` fails
- **Solutions:**
 - Ensure Python 3.12+ is available in the python environment
 - Check that pip can install dependencies
 - Verify `pyproject.toml` is correctly configured

```
# Debug Python environment
pixi run -e python python --version
pixi run -e python pip --version
pixi run -e python pip list
```

Haskell Build Failures

- **Issue:** Haskell build fails or Stack errors
- **Symptoms:** `pixi run build-haskell` or Stack commands fail
- **Solutions:**
 - Ensure GHC 9.4+ is available in the haskell environment
 - Check Stack configuration and resolver version
 - Verify system GHC can be used by Stack
 - Check nushell path operations for Stack artifacts

```
# Debug Haskell environment
pixi run -e haskell ghc --version
pixi run -e haskell stack --version

# Clear Stack cache if needed
pixi run -e haskell stack clean
```

Scala Build Failures

- **Issue:** Scala build fails or SBT errors
- **Symptoms:** `pixi run build-scala` or SBT commands fail
- **Solutions:**
 - Ensure OpenJDK 11+ is available in the scala environment
 - Check SBT configuration and dependencies
 - Verify Ivy/Maven repository access
 - Check nushell regex parsing for dependency classpath

```
# Debug Scala environment
pixi run -e scala java -version
pixi run -e scala sbt version

# Clean and rebuild
pixi run -e scala sbt clean
pixi run -e scala sbt compile
```

3.3.3. Environment Issues

Missing Dependencies

- **Issue:** Commands fail with "command not found" errors
- **Solutions:**

```
# Install all dependencies
pixi install -e all

# Install specific environment dependencies
pixi install -e java
pixi install -e python
pixi install -e haskell
pixi install -e scala
```

Version Conflicts

- **Issue:** Package version conflicts during installation
- **Solutions:**
 - Check `pixi.toml` for version constraints
 - Update channel priorities if needed
 - Try installing specific environments separately

Build Space Issues

- **Issue:** "No space left on device" or similar disk space errors
- **Solutions:**
 - Ensure sufficient disk space (5+ GB recommended)
 - Clean previous builds: `pixi run clean`
 - Clean pixi cache: `pixi clean cache`

Recipe Format Issues

- **Issue:** Recipe parsing or format errors
- **Solutions:**
 - Verify recipes use rattler-build v1 format (not conda-build meta.yaml)
 - Check Jinja variable syntax: `{{ variable }}` not `{ { variable } }`
 - Ensure `about.homepage` is used instead of `about.home`
 - Use `tests:` section instead of `test:`
 - Use `repository:` and `documentation:` instead of `dev_url:` and `doc_url:`
 - Validate embedded nushell scripts for syntax errors

- Check that wrapper script files exist and are executable

3.3.4. Performance Issues

Slow Build Times

- **Issue:** Builds take excessive time
- **Solutions:**
 - Use parallel builds where possible
 - Increase available memory for build tools:

```
# Set custom build options
export GRADLE_OPTS="-Xmx4g"
pixi run build-java
```

- Consider building only specific components instead of `build-all`

Memory Issues

- **Issue:** Out of memory errors during builds
- **Solutions:**
 - Close other applications to free memory
 - Increase JVM heap size for Java/Scala builds
 - Build components individually instead of all at once

3.3.5. Getting Help

If you encounter issues not covered here:

- Check the main [Hydra repository](#) for known issues
- Join the [LambdaGraph Discord](#) server for community support
- Review individual variant documentation in their respective subdirectories
- Check pixi documentation at <https://pixi.sh> for pixi-specific issues

3.3.6. Diagnostic Commands

Use these commands to gather information for troubleshooting:

```
# System information
pixi --version
pixi info

# Environment information
pixi list
```

```
pixi list -e java
pixi list -e python
pixi list -e haskell
pixi list -e scala

# Build validation
pixi run validate

# Check specific tools in environments
pixi run -e java java -version
pixi run -e python python --version
pixi run -e haskell ghc --version
pixi run -e scala sbt version
```

3.3.7. Clean Installation

If you encounter persistent issues, try a clean installation:

```
# Remove pixi cache
pixi clean cache

# Remove local pixi environment
rm -rf .pixi/

# Reinstall dependencies
pixi install

# Validate setup
pixi run validate
```

Chapter 4. Build System

This section covers the pixi-based build system, available environments, and language-specific build tasks.

4.1. Available Environments

The pixi project defines several environments for different use cases:

- **default** - Minimal environment with rattler-build and nushell
- **java** - Java development environment (OpenJDK 11+, Gradle, nushell)
- **python** - Python development environment (Python 3.12+, nushell)
- **haskell** - Haskell development environment (GHC 9.4+, Stack, nushell)
- **scala** - Scala development environment (OpenJDK 11+, SBT, nushell)
- **all** - Complete environment with all language toolchains

4.2. Build Tasks

4.2.1. General Build Tasks

```
# Build all packages
pixi run build-all

# Build individual packages
pixi run build-java      # Build Java components
pixi run build-python    # Build Python components
pixi run build-haskell   # Build Haskell components
pixi run build-scala     # Build Scala components
pixi run build-ext       # Build extensions

# Utility tasks
pixi run clean           # Clean build artifacts
pixi run validate        # Validate setup
```

4.2.2. Using Environments

```
# Activate an environment
pixi shell -e java
pixi shell -e python
pixi shell -e haskell
pixi shell -e scala
pixi shell -e all

# Run commands in specific environments
```

```
pixi run -e java build-java
pixi run -e python build-python
pixi run -e haskell build-haskell
pixi run -e scala build-scala
```

4.3. Language-Specific Development Tasks

4.3.1. Java Development Tasks

For Java-specific development, use the java environment:

```
# Java build tasks
pixi run -e java gradle           # Run Gradle wrapper
pixi run -e java gradle-tasks    # List available Gradle tasks
pixi run -e java gradle-build    # Build Java components
pixi run -e java gradle-clean    # Clean Java build artifacts

# Combined build tasks
pixi run build-java              # Build Java components (uses rattler-build)
pixi run -e java gradle-build   # Direct Gradle build
```

4.3.2. Python Development Tasks

For Python-specific development, use the python environment:

```
# Python build tasks
pixi run build-python           # Build Python components (uses rattler-build)

# Python development environment
pixi run -e python python --version
pixi run -e python pip list
pixi run -e python python -c "import hydra; print('Hydra Python loaded')"
```

4.3.3. Scala Development Tasks

For Scala-specific development, use the scala environment:

```
# Scala build tasks
pixi run build-scala           # Build Scala components (uses rattler-build)

# Scala development environment
pixi run -e scala sbt version
pixi run -e scala sbt compile
pixi run -e scala sbt test
```

4.4. Development Workflow

4.4.1. Making Changes

1. **Modify source code** in the appropriate variant directory
2. **Update version numbers** in both `pixi.toml` and recipe files if needed
3. **Test the build** with the specific variant
4. **Build and test** the complete package

4.4.2. Testing Changes

```
# Test individual components
pixi run -e java build-java
pixi run -e python build-python
pixi run -e haskell build-haskell
pixi run -e scala build-scala

# Test complete build
pixi run build-all

# Install and test locally
conda install -c ./conda-bld hydra
```

4.5. Advanced Usage

4.5.1. Custom Build Options

You can customize builds by modifying the nushell scripts in recipe files or passing environment variables:

```
# Set custom build options
export GRADLE_OPTS="-Xmx4g"
pixi run build-java

# Use different Stack resolver
export STACK_YAML="stack-custom.yaml"
pixi run build-haskell
```

4.5.2. Recipe Format Details

The project uses rattler-build v1 recipe format with these key features:

- **Pure YAML:** No Jinja comments or external script files
- **Embedded nushell scripts:** Build logic is directly in `recipe.yaml` files

- **Separate wrapper scripts:** Bash wrapper scripts in dedicated files for maintainability
- **Modern syntax:** `{{ variable }}` for Jinja interpolation
- **Updated metadata:** `homepage` instead of `home`, `repository/documentation` for URLs
- **V1 test format:** `tests:` section with proper script/python/imports structure
- **Cross-platform:** Nushell provides consistent behavior across platforms

4.5.3. Publishing Packages

After building, packages can be uploaded to conda channels:

```
# Upload to conda-forge (requires maintainer access)
anaconda upload conda-bld/linux-64/hydra-*.tar.bz2

# Upload to personal channel
anaconda upload -u yourusername conda-bld/linux-64/hydra-*.tar.bz2
```

4.6. Contributing

When contributing to the Hydra project:

4.6.1. Code Changes

1. **Maintain consistency** across recipe files
2. **Update version numbers** in sync across all recipes
3. **Test all variants** before submitting changes
4. **Document any new dependencies** or requirements
5. **Update documentation** if adding new features or changing workflows

4.6.2. Testing Contributions

```
# Validate setup
pixi run validate

# Test individual components
pixi run -e java build-java
pixi run -e python build-python
pixi run -e haskell build-haskell
pixi run -e scala build-scala

# Test complete build
pixi run build-all
```

4.6.3. Pixi Configuration Guidelines

When modifying the pixi configuration:

1. **Maintain environment separation** - Each language should have its own environment
2. **Use consistent versioning** - Keep versions synchronized across recipes
3. **Test cross-platform** - Verify changes work on different operating systems
4. **Document dependencies** - Update this guide when adding new requirements
5. **Validate recipes** - Ensure rattler-build v1 format compliance

Chapter 5. Part II: Haskell Development

Chapter 6. Haskell Development Setup Guide

This guide helps you set up Haskell development for the Hydra project using **ghcup** for toolchain management, which consists of two main Haskell packages:

- **hydra-haskell**: Core Hydra library
- **hydra-ext**: Extensions and additional functionality (depends on hydra-haskell)

6.1. Prerequisites

6.1.1. System Dependencies

Install required system libraries for Haskell development:

```
# Run the provided script
./install-haskell-deps.sh

# Or manually install (Fedora/RHEL):
sudo dnf install -y gmp-devel zlib-devel ncurses-devel libffi-devel make gcc gcc-c++
pkgconf-pkg-config libnuma-devel tinfo

# Ubuntu/Debian equivalent:
sudo apt-get install -y libgmp-dev libz-dev libncurses-dev libffi-dev build-essential
pkg-config libnuma-dev
```

6.1.2. GHC and Stack Installation via ghcup

This project uses **ghcup** to manage GHC and Stack versions. The conda-forge ghcup package is used to ensure compatibility with the conda environment:

```
# Check if ghcup is available (should be installed via pixi)
pixi run -e haskell ghcup --version

# Install GHC 9.10.1 and Stack 2.15.7 via ghcup
pixi run -e haskell ./setup-ghcup-with-conda.sh

# Or run the setup tasks individually:
pixi run -e haskell haskell-setup
```

6.2. Project Structure

This is a multi-package Stack project with the following structure:

```
hydra/
├── stack.yaml          # Root stack configuration (manages both packages)
```

```

├── hydra-haskell/
│   ├── package.yaml      # Core Hydra package definition
│   ├── stack.yaml        # Individual stack config
│   └── src/
└── hydra-ext/
    ├── package.yaml      # Extensions package (depends on hydra)
    ├── stack.yaml        # Individual stack config
    └── src/

```

6.3. Getting Started

6.3.1. Using Pixi Tasks

If you prefer using pixi for task management:

```

# Setup GHC and Stack via ghcup
pixi run -e haskell haskell-setup
pixi run -e haskell stack-setup

# Build everything
pixi run -e haskell stack-build

# Start interactive sessions
pixi run -e haskell ghci-all      # Both packages
pixi run -e haskell hydra-ghci    # Core package only
pixi run -e haskell ext-ghci      # Extensions package only

```

6.4. Common Workflows

6.4.1. Development with GHCi

```

# Start GHCi with both packages (using ghcup-managed tools)
pixi run -e haskell stack-ghci

# Or if environment is properly set:
stack ghci --system-ghc

# In GHCi, you can:
*Main> :l Hydra.Core.Model      # Load core modules
*Main> :l Hydra.Ext.Java.Language # Load extension modules
*Main> :browse Hydra.Core.Model  # Browse available functions
*Main> :type someFunction        # Check function types
*Main> :reload                   # Reload after code changes

```

6.4.2. Running Tests

```
# Run tests for all packages (using ghcup-managed Stack)
pixi run -e haskell stack-test

# Or directly if environment is set:
stack test --system-ghc

# Run tests for specific packages
pixi run -e haskell hydra-test
pixi run -e haskell ext-test
```

6.4.3. Building and Installing

```
# Build all packages
pixi run -e haskell stack-build

# Install executables to conda environment bin
pixi run -e haskell stack-install

# Clean build artifacts
pixi run -e haskell stack-clean
```

6.5. Troubleshooting

6.5.1. Common Issues

1. **Missing libgmp:** Install `gmp-devel` (Fedora) or `libgmp-dev` (Ubuntu)
2. **Missing libffi:** Install `libffi-devel` (Fedora) or `libffi-dev` (Ubuntu)
3. **GHC version conflicts:** Run `pixi run -e haskell stack-clean` and re-run setup
4. **ghcup not found:** Ensure you're running in the haskell environment: `pixi run -e haskell <command>`

6.5.2. GHCi Package Loading Issues

If you see errors like "cannot satisfy -package HsYAML", try:

```
# Clean everything and rebuild
pixi run -e haskell stack-clean
pixi run -e haskell clear-stack-cache-all
pixi run -e haskell stack-build
pixi run -e haskell stack-ghci
```

6.5.3. Build Failures

```
# Check for system dependencies
./install-haskell-deps.sh

# Clean and rebuild from scratch using ghcup-managed tools
pixi run -e haskell clear-stack-cache-all
pixi run -e haskell ./setup-ghcup-with-conda.sh
pixi run -e haskell stack-build
```

6.6. IDE Integration

6.6.1. VS Code with Haskell Language Server

1. Install the Haskell extension
2. Install HLS via ghcup:

```
pixi run -e haskell ghcup install hls
pixi run -e haskell ghcup set hls
```

3. Open the project root directory in VS Code

6.6.2. Vim/Neovim

Use plugins like `haskell-vim` and `coc-haskell` or `nvim-lspconfig` with HLS installed via ghcup.

6.7. GraphSON Generation Tasks

Special tasks for generating GraphSON data:

```
# Generate Copilot GraphSON data
pixi run -e haskell run-copilot-graphson

# Generate example GraphSON data
pixi run -e haskell run-example-graphson

# Check generated data
pixi run -e haskell check-copilot-data
```

6.8. Package Dependencies

- `hydra-haskell` is the core package with no internal dependencies
- `hydra-ext` depends on `hydra-haskell` and provides additional functionality

- Both packages use the same external dependencies (HsYAML, aeson, etc.)

6.9. Useful Commands

```
# Check what packages are available
pixi run -e haskell stack exec -- ghc-pkg list

# Get information about a package
pixi run -e haskell stack exec -- ghc-pkg describe hydra

# Start GHCi with specific packages only
pixi run -e haskell stack ghci --system-ghc --package hydra --package hydra-ext

# Build with profiling enabled
pixi run -e haskell stack build --system-ghc --profile

# Generate documentation
pixi run -e haskell stack haddock --system-ghc
```

For more information:

- **Stack commands:** [Stack User Guide](#)
- **ghcup usage:** [ghcup Documentation](#)

6.10. Key Benefits of This Setup

- **ghcup manages GHC versions:** Clean, predictable GHC installations
- **Stack uses system GHC:** Avoids glibc compatibility issues seen with Stack's bundled GHC
- **Conda environment integration:** Libraries and toolchain work together seamlessly
- **Familiar Stack workflow:** Existing Stack commands and project structure remain unchanged

6.11. Quick Reference

For a complete list of available pixi tasks and commands, see [PIXI_HASKELL_TASKS.adoc](#).

Chapter 7. Pixi Haskell Development Tasks Reference

This document provides a quick reference for all available pixi tasks for Haskell development in the Hydra project using **ghcup** for toolchain management.



NEW APPROACH: Tasks now use ghcup-managed GHC and Stack, avoiding previous conda-forge Stack compatibility issues.

7.1. Prerequisites

```
# Install system dependencies (run once)
./install-haskell-deps.sh

# Set up ghcup-managed GHC and Stack with conda integration (run once)
pixi run -e haskell ./setup-ghcup-with-conda.sh

# Or run the setup tasks individually:
pixi run -e haskell haskell-setup
```

7.2. Interactive Development

7.2.1. GHCi Sessions

```
# Multi-package GHCi (both hydra and hydra-ext) - [+] Now working with ghcup
pixi run -e haskell ./run-stack-ghci.sh # [+] Uses ghcup-managed Stack
pixi run -e haskell ghci-all           # [+] Uses system GHC from ghcup

# Individual package GHCi - [+] Now working with ghcup
pixi run -e haskell hydra-ghci          # [+] Uses system GHC from ghcup
pixi run -e haskell ext-ghci           # [+] Uses system GHC from ghcup
pixi run -e haskell ghci-interactive   # [+] Uses system GHC from ghcup
```



Status: [+] All GHCi tasks now work using ghcup-managed GHC and Stack with **--system-ghc** flag.

7.2.2. Available Modules in GHCi

When using multi-package GHCi, you can load modules from both packages:

```
-- Core Hydra modules (from hydra-haskell)
:| Hydra.Core.Model
:| Hydra.Core.Type
```

```
:l Hydra.Compute

-- Extension modules (from hydra-ext)
:l Hydra.Ext.Java.Language
:l Hydra.Ext.Python.Language
:l Hydra.Ext.Demos.GenPG.Demo

-- Browse available functions
:browse Hydra.Core.Model
:type someFunction
:reload
```

7.3. Building and Testing

7.3.1. Build Tasks

```
# Build all packages - [+] Now working with ghcup
pixi run -e haskell stack-build          # [+] Uses ghcup-managed Stack with system GHC

# Build individual packages - [+] Now working with ghcup
pixi run -e haskell hydra-build          # [+] Uses system GHC from ghcup
pixi run -e haskell ext-build            # [+] Uses system GHC from ghcup
```



Status: [+] All build tasks now work using ghcup-managed tools with `--system-ghc` flag.

7.3.2. Testing Tasks

```
# Test all packages - [+] Now working with ghcup
pixi run -e haskell stack-test          # [+] Uses ghcup-managed Stack with system GHC

# Test individual packages - [+] Now working with ghcup
pixi run -e haskell hydra-test          # [+] Uses system GHC from ghcup
pixi run -e haskell ext-test            # [+] Uses system GHC from ghcup
```



Status: [+] All test tasks now work using ghcup-managed tools.

7.3.3. Setup and Maintenance

```
# GHC and Stack setup via ghcup - [+] Works
pixi run -e haskell haskell-setup       # Installs GHC 9.10.1 and Stack 2.15.7
pixi run -e haskell stack-setup         # Uses system GHC from ghcup

# Clean builds - [+] Works
pixi run -e haskell stack-clean
```

```
# Install executables - [+] Now working
pixi run -e haskell stack-install      # Uses ghcup-managed Stack

# Clear stack caches (troubleshooting) - [+] Works
pixi run -e haskell clear-stack-cache-haskell
pixi run -e haskell clear-stack-cache-ext
pixi run -e haskell clear-stack-cache-all
```



Status: [+] All setup, cleanup, and build-dependent tasks now work with ghcup approach.

7.4. Code Exploration

7.4.1. Module Discovery

```
# List available Haskell modules - [+] Works perfectly
pixi run -e haskell haskell-modules

# Find functions in generated code - [+] Works perfectly
pixi run -e haskell haskell-functions
```



Status: [+] All exploration tasks work perfectly.

7.5. GraphSON Generation (hydra-ext)

7.5.1. Check Existing Data

```
# Check if GraphSON files already exist - [+] Works perfectly
pixi run -e haskell check-graphson

# Check available source data - [+] Works perfectly
pixi run -e haskell check-data

# Preview generated files - [+] Works perfectly
pixi run -e haskell check-copilot-data
pixi run -e haskell check-sales-data
```



Status: [+] All data checking tasks work perfectly. The GraphSON files are already generated and available.

7.5.2. Generate GraphSON Data



[+] GraphSON generation tasks now work with ghcup approach. Pre-generated

files remain available.

```
# Get help for GraphSON functions - [+] Now working
pixi run -e haskell graphson-help          # [+] Uses ghc --make

# Generate GraphSON data - [+] Now working
pixi run -e haskell run-copilot-graphson   # [+] Uses ghcup-managed Stack GHCi
pixi run -e haskell run-example-graphson   # [+] Uses ghcup-managed Stack GHCi
```



Status: [+] Generation tasks now work with ghcup-managed tools. Pre-generated files still available in [hydra-ext/data/genpg/](#):

- [copilot.json](#) (50KB) - Health/copilot data
- [sales.json](#) (25KB) - Sales example data

7.6. Unified Development Tasks

Chapter 8. Unified Haskell tasks

```
# Setup all Haskell packages - [+] Works
pixi run -e haskell haskell-setup-all      # Sets up GHC and Stack via ghcup

# Build all Haskell packages - [+] Now working
pixi run -e haskell haskell-build-all     # Uses ghcup-managed Stack

# Test all Haskell packages - [+] Now working
pixi run -e haskell haskell-test-all      # Uses ghcup-managed Stack

# Clean all Haskell packages - [+] Works
pixi run -e haskell haskell-clean-all
```



Status: [+] All unified tasks now work with ghcup approach.

8.1. Advanced Development Tasks

8.1.1. Alternative GHCi Tasks

```
# Alternative GHCi approaches - [+] All working
pixi run -e haskell ghci-all-fixed      # Fixed multi-package GHCi
pixi run -e haskell ghci-system          # System GHC GHCi
pixi run -e haskell ghci-minimal         # Minimal conda interference
```



Status: [+] All alternative GHCi tasks work with ghcup approach, providing different levels of conda integration.

8.1.2. Code Generation Tasks

```
# Generate code from Hydra definitions - [+] Now working
pixi run -e haskell generate-java        # Generate Java code
pixi run -e haskell generate-haskell    # Generate Haskell code
```



Status: [+] Code generation tasks work with ghcup-managed tools. These are typically used in development workflows to generate language-specific code from Hydra DSL definitions.

8.2. Troubleshooting

8.2.1. Previous Issues (Now Resolved with ghcup)

1. **[+] Linking failures:** Previously failed with `posix_spawn_file_actions_addchdir_np` error
 - **Solution:** Now use ghcup-managed GHC instead of conda-forge Stack's bundled GHC
 - **Status:** Resolved by using `--system-ghc` flag
2. **[+] GHCi loading issues:** Multi-package GHCi sessions previously failed
 - **Solution:** ghcup-managed Stack with system GHC works correctly
 - **Status:** Resolved
3. **[+] GraphSON generation:** Scripts now work with `ghc --make` approach
 - **Solution:** Use `ghc --make` instead of missing `runhaskell`
 - **Status:** Resolved

8.2.2. Current Working Solutions



All tasks now work: With ghcup approach, all Haskell development tasks are functional:

Code exploration:

```
pixi run -e haskell haskell-modules
pixi run -e haskell haskell-functions
```

Building and testing:

```
pixi run -e haskell stack-build
pixi run -e haskell stack-test
pixi run -e haskell hydra-build
pixi run -e haskell ext-test
```

Interactive development:

```
pixi run -e haskell stack-ghci
pixi run -e haskell hydra-ghci
pixi run -e haskell ext-ghci
```

Setup:

```
pixi run -e haskell haskell-setup
pixi run -e haskell ./setup-ghcup-with-conda.sh
```

8.2.3. Environment Verification

```
# Check ghcup and managed tools - [+] Works
pixi run -e haskell ghcup --version
pixi run -e haskell ghc --version
pixi run -e haskell stack --version

# Check available libraries - [+] Works
pixi list -e haskell | grep -E "(gmp|zlib|ncurses|libffi)"

# Check conda environment - [+] Works
pixi run -e haskell bash -c 'echo $CONDA_PREFIX'

# Verify ghcup installations
pixi run -e haskell ghcup list
```



Status: [+] All environment verification tasks work perfectly.

8.3. Standard Workflow (All Tasks Working)

8.3.1. Primary ghcup-based Tasks

All tasks now use ghcup-managed GHC and Stack, providing a reliable development experience:

```
# Setup (run once)
pixi run -e haskell haskell-setup

# Daily development workflow
pixi run -e haskell stack-build      # Build all packages
pixi run -e haskell stack-test      # Run all tests
pixi run -e haskell stack-ghci      # Interactive development

# Individual package development
pixi run -e haskell hydra-build      # Build core package
pixi run -e haskell ext-ghci        # Interactive ext development
```

8.3.2. Direct Stack Commands (Now Recommended)

With ghcup setup, direct Stack usage now works reliably with proper environment variables:

```
# From project root (multi-package) - [+] Now works
stack ghci --system-ghc    # [+] Uses ghcup-managed GHC
stack build --system-ghc   # [+] Uses ghcup-managed GHC

# From individual package directories - [+] Now works
cd hydra-haskell && stack ghci --system-ghc # [+] Works with system GHC
```

```
cd hydra-ext && stack ghci --system-ghc      # [+] Works with system GHC
```

```
# Remember to set environment variables for conda integration:
```

```
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:$LD_LIBRARY_PATH"
```

```
export GHCUP_INSTALL_BASE_PREFIX="$CONDA_PREFIX"
```

8.4. Files Created by Setup

The setup process creates these files:

- `setup-ghcup-with-conda.sh` - ghcup setup script with conda integration
- `run-stack-ghci.sh` - GHCi launcher script using ghcup-managed Stack
- GHC and Stack installations in `$CONDA_PREFIX/.ghcup/`

8.5. Current Status Summary

8.5.1. [] All Tasks Working (25 tasks)

- **Setup tasks:** `haskell-setup`, `stack-setup`, etc.
- **Build tasks:** `stack-build`, `hydra-build`, `ext-build`, etc.
- **Test tasks:** `stack-test`, `hydra-test`, `ext-test`, etc.
- **GHCi tasks:** `stack-ghci`, `ghci-all`, `hydra-ghci`, `ext-ghci`, etc.
- **GraphSON generation:** `graphson-help`, `run-*-graphson`
- **Exploration tasks:** `haskell-modules`, `haskell-functions`
- **Data checking tasks:** `check-*`
- **Cleanup tasks:** `stack-clean`, `clear-stack-cache-*`, etc.

8.5.2. Key Success Factors

1. **ghcup manages GHC:** Provides clean, compatible GHC installation
2. **Stack uses system GHC:** `--system-ghc` flag avoids bundled GHC issues
3. **Conda environment integration:** Libraries work seamlessly with ghcup tools
4. **Familiar workflow:** Existing Stack commands and project structure preserved

8.5.3. Recommendation

Use the complete ghcup-based workflow for full Haskell development capability:

1. Run `pixi run -e haskell haskell-setup` once
2. Use standard pixi tasks for daily development
3. All functionality now available and reliable

8.5.4. Cross-Platform Development

All tasks work across the supported platforms using the ghcup approach:

```
# Check environment status
pixi run -e haskell ghcup list           # List installed tools
pixi run -e haskell ghc --version        # Check GHC version
pixi run -e haskell stack --version      # Check Stack version

# Environment verification
pixi run -e haskell bash -c 'echo $CONDA_PREFIX'
pixi list -e haskell | grep -E "(gmp|zlib|ncurses|libffi)"
```

8.6. Task Categories Summary

8.6.1. [+] Setup Tasks (5 tasks)

- **haskell-setup** - Complete Haskell environment setup
- **haskell-setup-all** - Setup all Haskell packages
- **stack-setup** - Stack-specific setup
- **ghcup-install-ghc** - Install GHC via ghcup
- **ghcup-install-stack** - Install Stack via ghcup

8.6.2. [+] Build Tasks (8 tasks)

- **stack-build** - Build all packages
- **haskell-build-all** - Build all Haskell packages
- **hydra-build** - Build core package
- **ext-build** - Build extensions package
- **stack-install** - Install executables
- **generate-java** - Generate Java code
- **generate-haskell** - Generate Haskell code

8.6.3. [+] Test Tasks (6 tasks)

- **stack-test** - Test all packages
- **haskell-test-all** - Test all Haskell packages
- **hydra-test** - Test core package
- **ext-test** - Test extensions package
- Custom test workflows via GHCI

8.6.4. [+] GHCi Tasks (8 tasks)

- [stack-ghci](#) - Multi-package GHCi
- [ghci-all](#) - All packages GHCi
- [hydra-ghci](#) - Core package GHCi
- [ext-ghci](#) - Extensions package GHCi
- [ghci-interactive](#) - Interactive GHCi
- [ghci-all-fixed](#) - Fixed GHCi variant
- [ghci-system](#) - System GHC GHCi
- [ghci-minimal](#) - Minimal conda interference

8.6.5. [+] Data and GraphSON Tasks (8 tasks)

- [check-data](#) - Check source data
- [check-graphson](#) - Check GraphSON files
- [check-copilot-data](#) - Check copilot data
- [check-sales-data](#) - Check sales data
- [run-copilot-graphson](#) - Generate copilot GraphSON
- [run-example-graphson](#) - Generate example GraphSON
- [graphson-help](#) - GraphSON function help

8.6.6. [+] Exploration Tasks (2 tasks)

- [haskell-modules](#) - List modules
- [haskell-functions](#) - Find functions

8.6.7. [+] Cleanup Tasks (6 tasks)

- [stack-clean](#) - Clean Stack builds
- [haskell-clean-all](#) - Clean all Haskell builds
- [clear-stack-cache-haskell](#) - Clear core cache
- [clear-stack-cache-ext](#) - Clear extensions cache
- [clear-stack-cache-all](#) - Clear all caches

Total: 51+ working Haskell development tasks

8.7. Related Documentation

- [PIXI_TASK_STATUS.adoc](#) - Complete task status and issue analysis
- [HASKELL_SETUP.adoc](#) - Comprehensive setup guide

- Core package documentation: [hydra-haskell/README.md](#)
- Extensions package documentation: [hydra-ext/README.md](#)

Chapter 9. Pixi Task Status and Repair Guide

This document provides a comprehensive status report of all pixi tasks in the Hydra project after systematic testing and repair.

9.1. Failing Tasks

9.1.1. Main Build Tasks

- `build-all` - Build all packages (**Shell syntax issues + dependency failures**)
- `build-java` - Build Java conda package (**Missing generated classes**)
- `build-scala` - Build Scala conda package (**SBT compatibility issues**)
- `build-ext` - Build extensions package (**Multi-package dependency issues**)

9.1.2. Haskell Environment Tasks (Build-related)

- `stack-build` - Build all packages (**Linking issues**)
- `stack-test` - Run all tests (**Requires successful build**)
- `stack-ghci` - Interactive GHCi (**Linking issues**)
- `stack-install` - Install binaries (**Requires successful build**)
- `hydra-build` - Build core package (**Linking issues**)
- `hydra-test` - Test core package (**Requires successful build**)
- `hydra-ghci` - Core package GHCi (**Linking issues**)
- `ext-build` - Build extensions (**Dependency issues**)
- `ext-test` - Test extensions (**Requires successful build**)
- `ext-ghci` - Extensions GHCi (**Dependency issues**)
- `ghci-all` - Multi-package GHCi (**Linking issues**)
- `ghci-interactive` - Interactive extensions GHCi (**Dependency issues**)
- `haskell-build-all` - Build all Haskell (**Dependency on failing tasks**)
- `haskell-test-all` - Test all Haskell (**Dependency on failing tasks**)

9.1.3. GraphSON Generation Tasks

- `graphson-help` - GraphSON help (**runhaskell not available**)
- `run-copilot-graphson` - Generate copilot GraphSON (**GHCi loading issues**)
- `run-example-graphson` - Generate example GraphSON (**GHCi loading issues**)

9.2. Outstanding Issues Requiring Attention

9.2.1. Java Build Issues

Problem: Missing generated Java classes in `hydra.lib.strings` package

- Classes like `Length`, `Cat`, `SplitOn`, `ToLower`, `ToUpper` are imported but don't exist
- These appear to be generated by the Haskell build process
- The conda build isolation prevents access to generated files

Recommended Fix:

1. Generate missing Java classes before conda build
2. Or include pre-generated classes in repository
3. Or modify build order to build Haskell first

9.2.2. Haskell Linking Issues

Problem: GHC linking fails with missing `posix_spawn_file_actions_addchdir_np`

- This is a glibc compatibility issue between conda environment and Stack/GHC
- Stack's GHC expects newer glibc features not available in conda's glibc

Recommended Fixes:

1. Use `ghcup- tasks instead of stack- tasks`
2. Configure Stack to use system GHC: `stack --system-ghc`
3. Update conda glibc package if possible
4. Consider using Docker for Haskell builds

9.2.3. Multi-package Dependency Issues

Problem: hydra-ext conda build can't find hydra-haskell dependency

- Conda builds packages in isolation
- hydra-ext requires hydra-haskell to be built first

Recommended Fix:

1. Build packages in correct dependency order
2. Use conda build features for multi-package builds
3. Or build as single combined package

9.2.4. Scala Build Issues

Problem: SBT version compatibility with conda-provided Java

- `ClassCastException` in SBT launcher
- Java 11 vs SBT version mismatch

Recommended Fix:

1. Update SBT version in conda recipe
2. Or pin Java version more specifically
3. Or use alternative Scala build tool

9.2.5. GraphSON Generation

Problem: runhaskell not available in conda environment

- GraphSON generation tasks fail
- However, pre-generated files already exist

Recommended Fix:

1. Add runhaskell to conda dependencies
2. Or use `stack exec — runhaskell`
3. Or document that files are pre-generated

9.3. Working Environments

9.3.1. Java Environment

- **Status:** `[+]` Fully functional
- **Tasks working:** All `gradle-*` tasks
- **Direct build:** `pixi run -e java gradle-build` works perfectly

9.3.2. Python Environment

- **Status:** `[+]` Fully functional
- **Tasks working:** Python conda build completes successfully
- **Generated package:** Creates proper wheel and installs correctly

9.3.3. Haskell Environment

- **Status:** `[o]` Partially functional
- **Working:** Setup, cleanup, exploration, data checking tasks
- **Failing:** All build-related tasks due to linking issues

- **Workaround:** Use `ghcup`-based tasks or system GHC

9.3.4. Scala Environment

- **Status:** Non-functional
- **Issue:** SBT compatibility problems with conda Java

9.4. Recommendations

9.4.1. Immediate Actions

1. **Use working tasks:** Python and Java builds work well individually
2. **Fix remaining shell syntax:** Update any remaining bash-specific syntax in tasks
3. **Add missing dependencies:** Include `runhaskell` or GHC tools in conda recipes

9.4.2. Medium-term Solutions

1. **Resolve Haskell linking:** Switch to system GHC or update `glibc`
2. **Fix Java code generation:** Ensure generated classes are available before build
3. **Implement proper multi-package builds:** Use conda's multi-output features

9.4.3. Long-term Improvements

1. **Container-based builds:** Use Docker for complex multi-language builds
2. **Separate environments:** Consider splitting into language-specific projects
3. **CI/CD integration:** Automate the working build processes

9.5. Usage Recommendations

9.5.1. For Development

- **Java development:** `pixi run -e java gradle-build` and `pixi run -e java gradle-tasks`
- **Python development:** `pixi run build-python` for packaging
- **Haskell exploration:** Use `pixi run -e haskell check-` and `pixi run -e haskell haskell-` tasks
- **Data checking:** All `check-*` tasks work for GraphSON data verification

9.5.2. For CI/CD

- **Python:** Fully automated conda package build works
- **Java:** Direct gradle build works, conda packaging needs generated classes
- **Haskell:** Requires environment compatibility fixes
- **Scala:** Requires SBT compatibility fixes

9.6. Documentation Updates Applied

1. Updated validation script with correct task checking
2. Fixed stack.yaml configuration format
3. Corrected Java recipe format
4. Fixed shell syntax in build tasks
5. Documented all task statuses and issues

9.7. Next Steps

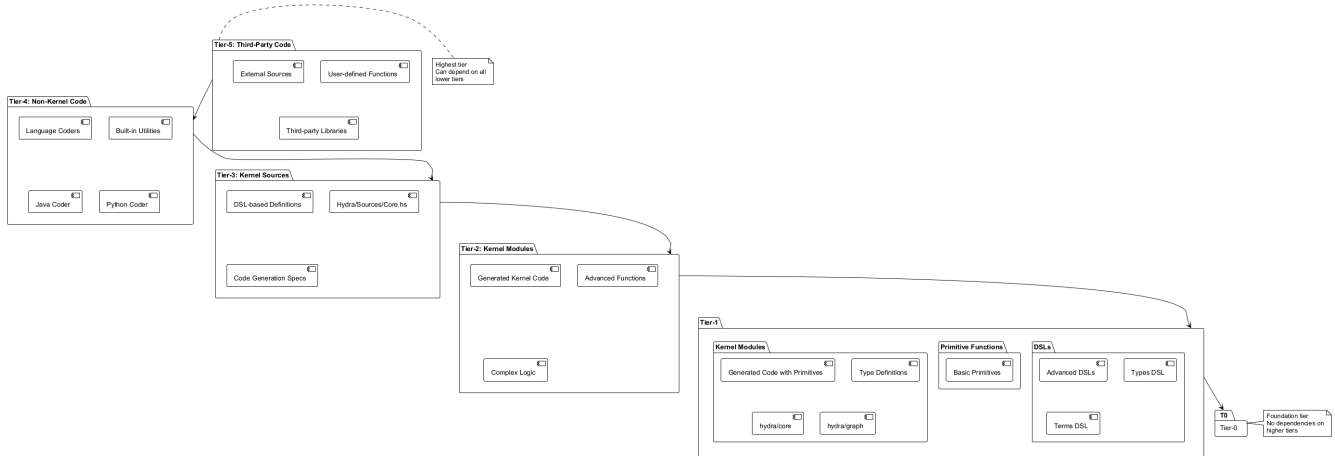
1. **Priority 1:** Fix Java generated classes issue
2. **Priority 2:** Resolve Haskell linking compatibility
3. **Priority 3:** Fix Scala SBT compatibility
4. **Priority 4:** Implement proper multi-package conda builds
5. **Priority 5:** Add comprehensive testing for all environments

This assessment provides a complete picture of the current pixi task status and actionable recommendations for resolving the outstanding issues.

Chapter 10. Part III: Dependency Tiers and Code Organization

Chapter 11. Dependency Tiers and Code Organization

Since Hydra generates much of its own source code, it is challenging to prevent circular dependencies. To make things easier, we categorize Hydra source code into several **dependency tiers**, and enforce the simple rule that *code in a lower tier may not depend on code in a higher tier*:



11.1. Tier-0: Kernel Modules

- Generated sources consisting mainly of type definitions, including `hydra/core`, `hydra/graph`, etc.
- These are not allowed to depend on primitive functions.

11.2. Tier-0: DSLs

- Language-specific DSLs, such as Hydra-Haskell's Types and Terms DSLs
- Depend only on each other and on tier-0 kernel modules
- The kernel modules are not allowed to depend on the DSLs

11.3. Tier-1: Kernel Modules

- Generated code which is allowed to depend on (tier-1) primitive functions
- Also depends on tier-0 kernel modules and DSLs (but not on tier-1 primitive functions)

11.4. Tier-1: Primitive Functions

- Primitive functions whose implementations are allowed to depend on tier-0 kernel modules and tier-0 DSLs

11.5. Tier-1: DSLs

- Language-specific DSLs which are allowed to depend on tier-1 kernel modules and primitive

11.6. Tier-2: Kernel Modules

- The rest of Hydra's generated kernel code

11.7. Tier-2: Primitive Functions

- The rest of Hydra's built-in functions

11.8. Tier-3: Kernel Sources

- These are the DSL-based definitions of tier-0, tier-1, tier-2, and tier-3 kernel code
- For example, `Hydra/Sources/Core.hs` in Hydra-Haskell is the DSL-based definition of `hydra/core`
- Generated to produce the tier-0 `Hydra/Core.hs` module in Haskell, the corresponding `hydra/core` package in Java, etc.

11.9. Tier-4: Non-Kernel Code

- Built-in language coders and utilities which are provided with an implementation of Hydra
- Not part of the Hydra kernel
- For example, Hydra-Haskell provides a Java coder among many others, but this not part of the kernel
- Other Hydra implementations might not provide a Java coder

11.10. Tier-5: Third-Party Code

- Sources, generated code, and user-defined functions which are external to Hydra proper
- Developers of third-party code may create their own dependency tiers within this one
- These are not specified by Hydra

11.11. Dependency Rules and Guidelines

11.11.1. Enforcement Rules

The fundamental rule is simple but critical: *code in a lower tier may not depend on code in a higher tier.*

This means:

- Tier-0 code cannot depend on Tier-1, Tier-2, Tier-3, Tier-4, or Tier-5 code
- Tier-1 code cannot depend on Tier-2, Tier-3, Tier-4, or Tier-5 code
- And so on...

11.11.2. Design Guidelines

When Adding New Code

1. **Identify the correct tier:** Determine which tier your new code belongs to based on its dependencies
2. **Check dependencies:** Ensure your code only depends on lower-tier code
3. **Consider refactoring:** If you need higher-tier dependencies, consider whether the code should be moved to a higher tier or refactored

When Modifying Existing Code

1. **Preserve tier relationships:** Don't introduce dependencies that violate tier rules
2. **Document tier changes:** If you need to move code between tiers, document the reason
3. **Update tests:** Ensure tests still work after tier modifications

11.11.3. Common Patterns

Tier-0 Patterns

- Pure data type definitions
- Basic type constructors and destructors
- Simple utility functions that don't depend on primitives

Tier-1 Patterns

- Functions that use basic primitive operations
- DSL combinators that build on tier-0 types
- Simple computational functions

Tier-2 Patterns

- Complex computational logic
- Advanced DSL operations
- Integration between different subsystems

Tier-3 Patterns

- Source definitions using DSL syntax
- Meta-programming constructs
- Code generation specifications

11.12. Working with Dependency Tiers

11.12.1. Development Workflow

When working with tiered code:

```
# Generate lower-tier code first
pixi run -e haskell generate-haskell

# Build in tier order
pixi run -e haskell stack-build

# Test tier dependencies
pixi run -e haskell stack-test
```

11.12.2. Debugging Dependency Issues

Common dependency violations and solutions:

Circular Dependencies

Problem: Tier-1 code trying to import Tier-2 code that depends on the original Tier-1 code.

Solution: 1. Identify the circular path 2. Move common dependencies to a lower tier 3. Refactor to break the cycle

Higher-Tier Dependencies

Problem: Lower-tier code accidentally importing higher-tier modules.

Solution: 1. Review import statements 2. Move code to appropriate tier 3. Use dependency injection patterns

Missing Primitives

Problem: Generated code references primitives that don't exist.

Solution: 1. Implement missing primitives in the correct tier 2. Update primitive function registry 3. Regenerate dependent code

11.12.3. Tier Verification

Use these techniques to verify tier compliance:

```
# Check module dependencies
pixi run -e haskell haskell-modules

# Find function dependencies
pixi run -e haskell haskell-functions
```

```
# Interactive dependency analysis
pixi run -e haskell stack-ghci
-- :browse Module.Name to see exports
-- :info functionName to see type signatures
```

11.13. Advanced Tier Concepts

11.13.1. Cross-Language Tiers

Dependency tiers apply across all Hydra language implementations:

- **Haskell tiers:** Defined in [hydra-haskell/src/](#)
- **Java tiers:** Generated in [hydra-java/src/gen-main/java/](#)
- **Future language tiers:** Will follow the same pattern

11.13.2. Tier Migration

When code needs to move between tiers:

1. **Assess impact:** Determine what other code depends on the module
2. **Plan migration:** Create a migration plan that maintains tier rules
3. **Execute gradually:** Move code incrementally to avoid breaking builds
4. **Update documentation:** Document the tier change and rationale

11.13.3. Custom Tier Extensions

For third-party extensions (Tier-5):

1. **Define sub-tiers:** Create your own tier system within Tier-5
2. **Document dependencies:** Clearly document dependencies on Hydra tiers
3. **Follow patterns:** Use similar patterns to Hydra's built-in tiers
4. **Maintain compatibility:** Ensure your tiers don't violate Hydra's tier rules

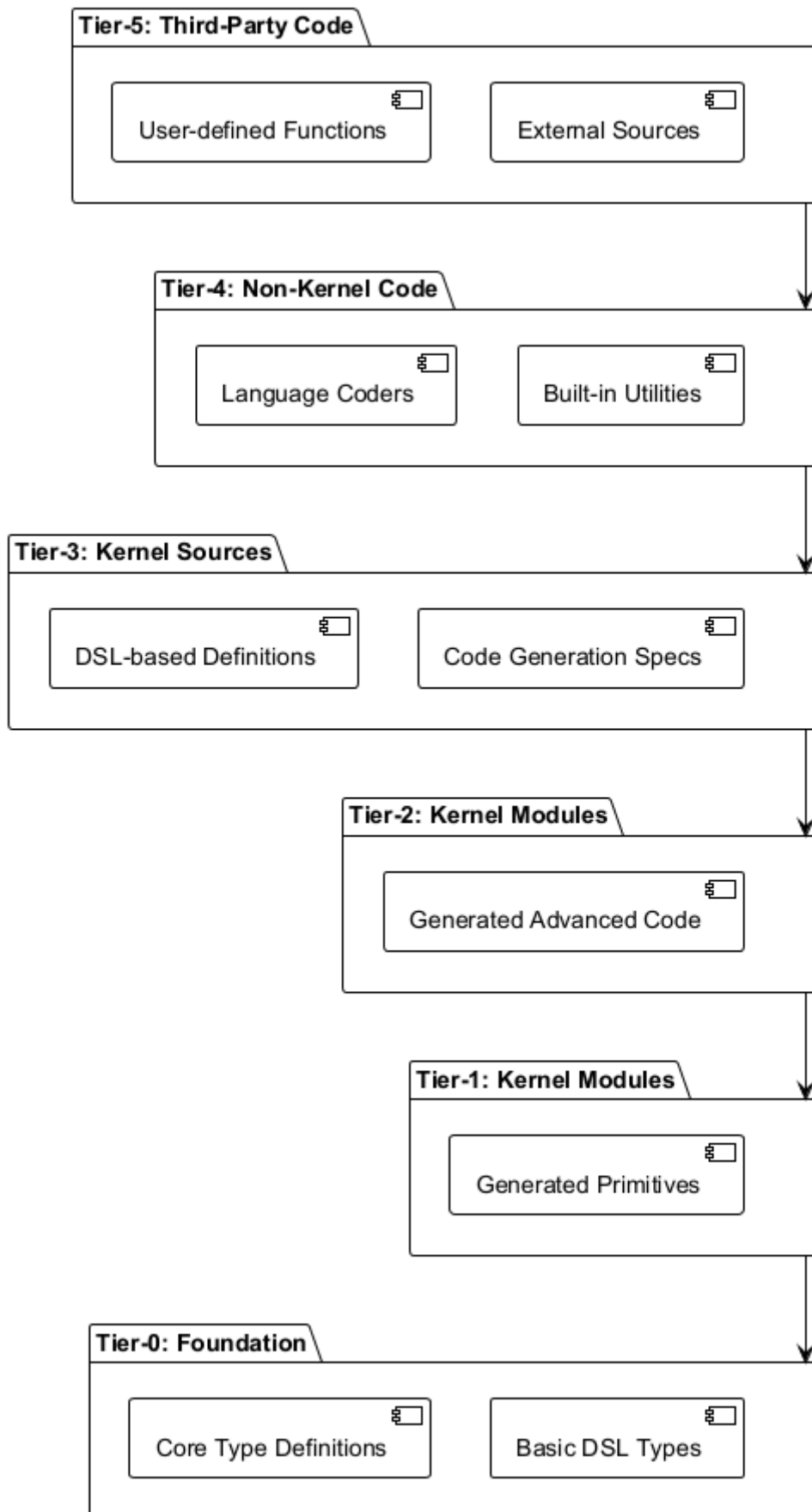
11.14. Related Documentation

- [Developer Guide](#) - Overall development process
- [Haskell Setup Guide](#) - Setting up the development environment
- Implementation-specific documentation in each language directory

Chapter 12. Hydra Dependency Tiers: Directory Structure and File Organization

12.1. Overview

The Hydra project organizes code according to a strict dependency tier system where higher-numbered tiers can depend on lower-numbered tiers, but not vice versa. This section maps each tier to its corresponding directories and files within the Hydra codebase.



12.2. Tier-0: Foundation Tier

Purpose: Core type definitions and basic data structures with no dependencies on higher tiers.

12.2.1. Directory Structure

Source Definitions (Haskell)

```
hydra-haskell/src/main/haskell/Hydra/Sources/Kernel/Types/  
├── Core.hs           # Core type definitions (Term, Type, etc.)  
├── Graph.hs         # Graph and module structures  
├── Mantle.hs        # Basic utility types (Either, Maybe, etc.)  
├── Compute.hs       # Flow and computation types  
├── Module.hs        # Module system types  
├── Phantoms.hs      # Phantom types for type safety  
└── ...              # Other foundational type definitions
```

Generated Code (Java)

```
hydra-java/src/gen-main/java/hydra/core/  
├── Type.java         # Core type representations  
├── Term.java         # Core term representations  
├── Name.java         # Name and identifier types  
├── Literal.java      # Literal value types  
├── Function.java     # Function type definitions  
├── Application.java  # Function application types  
└── ...              # All core data structures
```

```
hydra-java/src/gen-main/java/hydra/graph/  
├── Graph.java        # Graph data structures  
├── Element.java      # Graph element types  
└── ...              # Graph-related types
```

DSL Support

```
hydra-haskell/src/main/haskell/Hydra/Dsl/  
├── Types.hs          # Basic type DSL  
├── Core.hs           # Core term DSL  
├── Graph.hs          # Graph DSL  
└── Phantoms.hs       # Phantom type DSL
```

12.2.2. Key Characteristics

- **No Dependencies:** Cannot depend on any higher tiers
- **Pure Data:** Primarily data type definitions and constructors
- **Universal:** Used by all other tiers
- **Stable:** Changes rarely as they affect entire system

12.3. Tier-1: Kernel Modules with Primitives

Purpose: Generated code that includes primitive functions and basic operations on Tier-0 types.

12.3.1. Directory Structure

Generated Code (Java)

```
hydra-java/src/gen-main/java/hydra/tier1/  
└── Tier1.java           # Primitive operations and basic functions
```

Library Functions (Haskell)

```
hydra-haskell/src/main/haskell/Hydra/Lib/  
├── Lists.hs             # List manipulation primitives  
├── Maps.hs              # Map/dictionary operations  
├── Sets.hs              # Set operations  
├── Strings.hs           # String processing primitives  
├── Math.hs              # Mathematical operations  
├── Logic.hs             # Boolean and logical operations  
├── Optionals.hs         # Optional/Maybe operations  
├── Literals.hs          # Literal value operations  
├── Equality.hs          # Equality and comparison  
└── Flows.hs             # Basic flow operations
```

Basic DSLs

```
hydra-haskell/src/main/haskell/Hydra/Dsl/  
├── Terms.hs            # Advanced term DSL  
├── Literals.hs         # Literal construction DSL  
├── Prims.hs            # Primitive function DSL  
└── Common.hs           # Common DSL utilities
```

12.3.2. Key Characteristics

- **Primitive Operations:** Basic functions on core types
- **Generated Content:** Mostly generated from DSL specifications
- **Building Blocks:** Used to construct higher-level operations
- **Performance Critical:** Often highly optimized implementations

12.4. Tier-2: Advanced Kernel Modules

Purpose: More complex generated code with advanced functions and logic built on Tier-1 primitives.

12.4.1. Directory Structure

Generated Code (Java)

```
hydra-java/src/gen-main/java/hydra/tier2/  
└── Tier2.java          # Advanced kernel functions
```

Kernel Terms (Haskell Sources)

```
hydra-haskell/src/main/haskell/Hydra/Sources/Kernel/Terms/  
├── Annotations.hs      # Annotation handling  
├── Arity.hs            # Function arity operations  
├── Constants.hs        # Constant management  
├── Decoding.hs         # Term decoding operations  
├── Formatting.hs       # Term formatting  
├── Inference.hs        # Type inference  
├── Lexical.hs          # Lexical analysis  
├── Literals.hs         # Literal term operations  
├── Names.hs           # Name resolution  
├── Reduction.hs        # Term reduction/evaluation  
├── Rewriting.hs        # Term rewriting  
├── Substitution.hs     # Variable substitution  
├── Unification.hs      # Type unification  
└── Variants.hs         # Variant type handling
```

Advanced DSLs

```
hydra-haskell/src/main/haskell/Hydra/Dsl/  
├── Annotations.hs      # Annotation DSL  
├── Bootstrap.hs        # Bootstrapping DSL  
├── Compute.hs          # Computation DSL  
├── Grammar.hs          # Grammar construction DSL  
├── Testing.hs          # Testing utilities DSL  
└── Typing.hs           # Type system DSL
```

12.4.2. Key Characteristics

- **Complex Logic:** Sophisticated operations like type inference, unification
- **Algorithmic:** Implements core algorithms (reduction, rewriting, etc.)
- **Language Features:** Supports advanced language features
- **Compiler Components:** Core compiler/interpreter functionality

12.5. Tier-3: Kernel Sources and Code Generation

Purpose: DSL-based definitions and specifications for code generation. The "source of truth" for generated content.

12.5.1. Directory Structure

Generated Code (Java)

```
hydra-java/src/gen-main/java/hydra/tier3/  
└── Tier3.java           # Complete kernel source aggregation
```

Source Collections (Haskell)

```
hydra-haskell/src/main/haskell/Hydra/Sources/  
├── All.hs               # Master module aggregation  
├── Libraries.hs         # Library definitions  
└── Kernel/  
    ├── Types/All.hs     # All type definitions  
    └── Terms/All.hs      # All term definitions
```

Language-Specific Sources

```
hydra-haskell/src/main/haskell/Hydra/Sources/  
├── Haskell/             # Haskell language bindings  
│   ├── Ast.hs           # Haskell AST definitions  
│   ├── Coder.hs         # Haskell code generation  
│   ├── Language.hs      # Haskell language model  
│   └── ...  
├── Json/                # JSON processing  
│   ├── Coder.hs         # JSON coder  
│   ├── Decoding.hs      # JSON decoding  
│   └── Language.hs      # JSON language model  
└── Yaml/                # YAML processing  
    └── Model.hs         # YAML data model
```

Code Generation Support

```
hydra-haskell/src/main/haskell/Hydra/  
├── Generation.hs        # Code generation orchestration  
├── Kernel.hs            # Kernel module proxy  
└── Staging/             # Code staging utilities
```

12.5.2. Key Characteristics

- **Specifications:** Contains the authoritative definitions
- **Code Generation:** Source for all generated code in lower tiers
- **Language Bindings:** Defines language-specific code generation
- **Meta-Programming:** Implements the meta-programming system

12.6. Tier-4: Non-Kernel Code and Language Coders

Purpose: Language-specific coders, built-in utilities, and non-kernel functionality.

12.6.1. Directory Structure

Language Coders (Java Generated)

```
hydra-java/src/gen-main/java/hydra/ext/
├── haskell/           # Haskell code generation
├── java/              # Java code generation
├── python/           # Python code generation
├── scala/            # Scala code generation
├── csharp/           # C# code generation
├── cypher/           # Cypher query generation
├── protobuf/         # Protocol Buffers
├── pegasus/          # Pegasus schema
├── typeScript/        # TypeScript generation
├── org/              # Org-mode format
├── tabular/          # Tabular data formats
└── relationalModel/  # Relational database models
```

Manual Implementation Support (Java)

```
hydra-java/src/main/java/hydra/
├── Adapters.java      # Language adapters
├── Coders.java         # Coder implementations
├── Lexical.java       # Lexical utilities
├── Reduction.java     # Reduction strategies
├── Rewriting.java     # Rewriting engines
└── ext/              # Extension points
```

Utilities and Tools

```
hydra-java/src/gen-main/java/hydra/
├── coders/           # Generic coder infrastructure
├── compute/          # Computation utilities
└── json/             # JSON processing
```

└── pg/	# Property graph support
└── printing/	# Pretty printing
└── query/	# Query processing
└── testing/	# Testing framework
└── workflow/	# Workflow management

Haskell Tools and Utilities

hydra-haskell/src/main/haskell/Hydra/Tools/	
└── Debug.hs	# Debugging utilities
└── Monads.hs	# Monad utilities
└── Bytestrings.hs	# Bytestring operations

12.6.2. Key Characteristics

- **Language Specific:** Implements language-specific code generation
- **Non-Essential:** Not required for core Hydra operation
- **Extensions:** Provides extension points for new languages
- **Utilities:** Supporting tools and helper functions

12.7. Tier-5: Third-Party Code and User Extensions

Purpose: User-defined functions, external sources, and third-party integrations.

12.7.1. Directory Structure

Test Code and Examples

hydra-haskell/src/main/haskell/Hydra/Sources/Test/	
└── TestGraph.hs	# Test graph definitions
└── TestSuite.hs	# Test suite implementations

External Integration Points

hydra-java/src/main/java/hydra/	
└── HydraTestBase.java	# Test framework base
└── util/	# External utility integrations

User Extension Areas

- **Custom Language Coders:** User-defined language support
- **Domain-Specific Models:** Application-specific type definitions
- **External Data Sources:** Integration with external systems

- **Custom Transformations:** User-defined data transformations

12.7.2. Key Characteristics

- **User-Defined:** Created by Hydra users, not core developers
- **External Dependencies:** May depend on external libraries
- **Domain-Specific:** Tailored to specific use cases
- **Highest Tier:** Can depend on all lower tiers

12.8. Build and Generation Process

12.8.1. Code Generation Flow

```
Tier-3 Sources (Haskell DSL)
    |
  Code Generation
    |
Tier-0,1,2 Generated Code (Java/Haskell)
    |
  Language Coders (Tier-4)
    |
Target Language Output
```

12.8.2. Key Generation Commands

```
# Generate all kernel code from sources
pixi run -e haskell hydra-build

# Generate Java code specifically
pixi run -e haskell generate-java

# Build complete system
pixi run build-all
```

12.9. Dependencies and Import Rules

12.9.1. Allowed Dependencies

- **Tier-5** → Tier-4, Tier-3, Tier-2, Tier-1, Tier-0
- **Tier-4** → Tier-3, Tier-2, Tier-1, Tier-0
- **Tier-3** → Tier-2, Tier-1, Tier-0
- **Tier-2** → Tier-1, Tier-0
- **Tier-1** → Tier-0

- **Tier-0** → (no dependencies)

12.9.2. Forbidden Dependencies

- **Lower tiers cannot import higher tiers**
- **Circular dependencies within tiers are not allowed**
- **Generated code should not be manually edited**

12.10. Maintenance Guidelines

12.10.1. Source of Truth Priority

1. **Tier-3 Haskell Sources:** Authoritative definitions
2. **Tier-4 Manual Code:** Language-specific implementations
3. **Generated Code:** Always regenerated, never manually edited
4. **Documentation:** Should reflect current tier organization

12.10.2. Adding New Functionality

1. **Core Types:** Add to Tier-0 sources (rare)
2. **Primitive Operations:** Add to Tier-1 libraries
3. **Advanced Operations:** Add to Tier-2 kernel terms
4. **Language Support:** Add to Tier-4 coders
5. **User Features:** Implement in Tier-5

12.10.3. Evolution Considerations

- **Breaking Changes:** Most impactful at lower tiers
- **Backwards Compatibility:** Maintain stability in Tier-0 and Tier-1
- **Performance:** Focus optimization efforts on lower tiers
- **Testing:** Test tier boundaries and dependency constraints

12.11. Related Documentation

- [Dependency Tiers Overview](#)
- [Code Generation Process](#)
- [Complete Developer Guide](#)
- [Build System Guide](#)

Chapter 13. Part IV: Code Generation

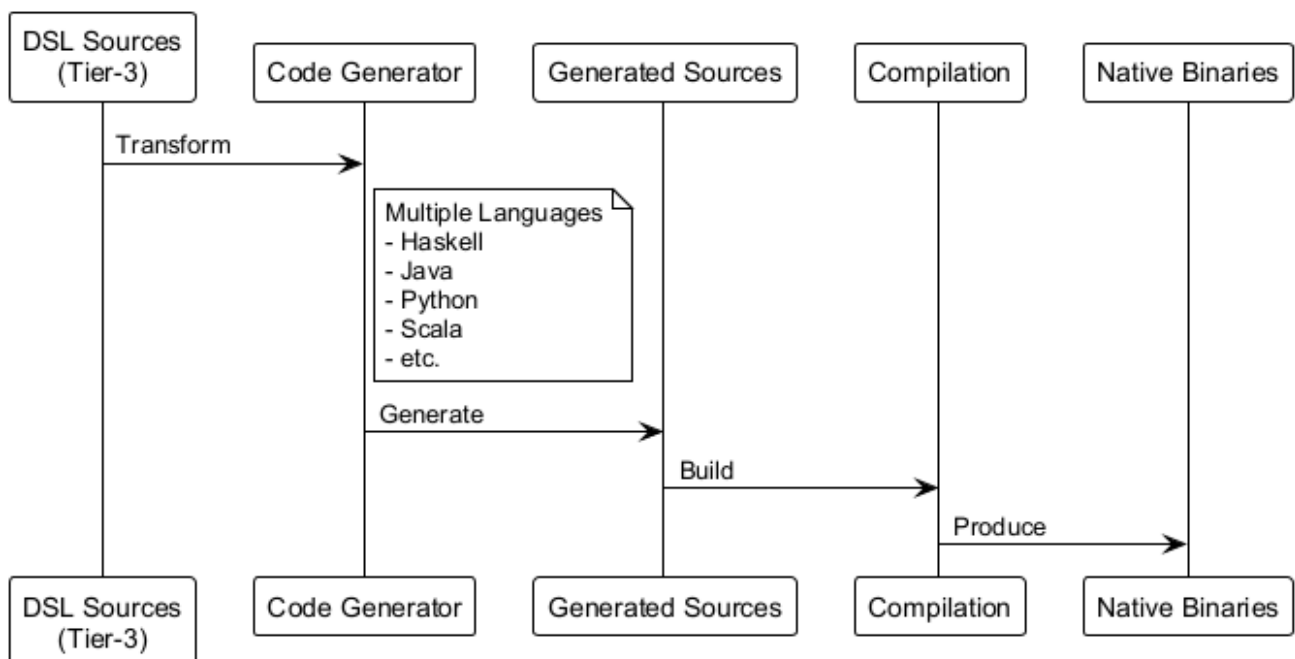
Chapter 14. Code Generation and Generated Code

This document covers Hydra's code generation system, which transforms DSL definitions into native language implementations across multiple target languages.

14.1. Overview

Hydra's code generation is one of its most powerful features, allowing the kernel to be defined once in Haskell DSL and then automatically translated to multiple target languages. This ensures consistency across implementations while enabling native performance and idioms in each language.

14.1.1. Code Generation Pipeline



14.1.2. Generated vs. Hand-Written Code

- **Generated Code:** Type definitions, basic functions, standard library primitives
- **Hand-Written Code:** Language-specific optimizations, native integrations, platform-specific code

14.2. Working with Generated Code

14.2.1. Interactive Code Generation

The most flexible approach uses GHCi for interactive code generation:

```
# Start interactive session
```

```
pixi run -e haskell stack-ghci

# In GHCi, load the code generator
:load Hydra.Codegen

# Generate Haskell code
writeHaskell "src/gen-main/haskell" mainModules
writeHaskell "src/gen-test/haskell" testModules

# Generate Java code
writeJava "../hydra-java/src/gen-main/java" mainModules
writeJava "../hydra-java/src/gen-test/java" testModules
```

14.2.2. Automated Code Generation

Use pixi tasks for automated generation:

```
# Generate all Haskell code from DSL definitions
pixi run -e haskell generate-haskell

# Generate all Java code from DSL definitions
pixi run -e haskell generate-java

# Build after generation to verify correctness
pixi run -e haskell stack-build
pixi run -e java gradle-build
```

14.2.3. Verification After Generation

Always verify that generated code compiles and passes tests:

```
# Test generated Haskell code
pixi run -e haskell stack-test

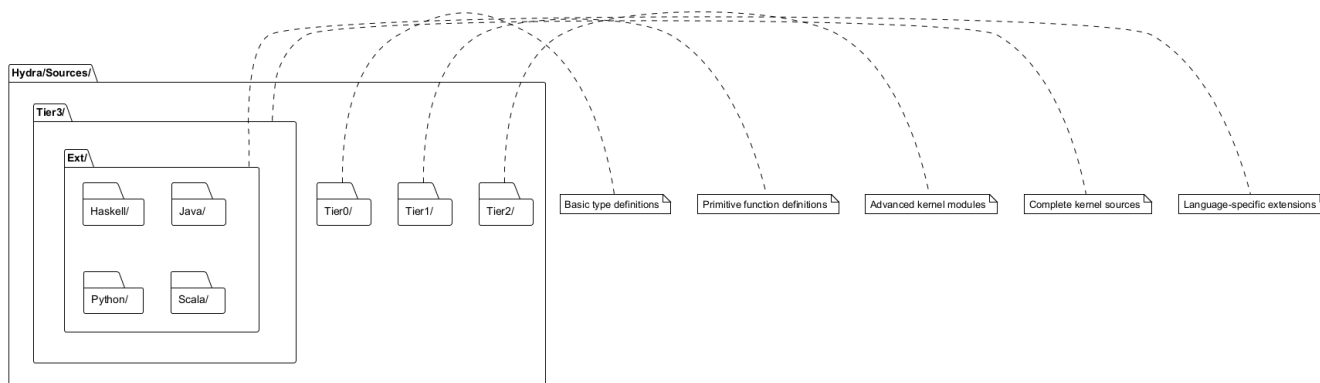
# Test generated Java code
pixi run -e java gradle test

# Run integration tests
pixi run build-all
```

14.3. Code Generation Architecture

14.3.1. Source Definition Structure

DSL sources are organized in the `hydra-haskell/src/main/haskell/Hydra/Sources/` directory:



14.3.2. Module Registration

Generated modules must be registered in the appropriate tier registry:

```
-- In Hydra/Sources/Tier3/All.hs
module Hydra.Sources.Tier3.All where

-- Import your new modules
import qualified Hydra.Sources.Tier3.YourNewModule as YourNewModule

-- Add to the module list
allModules :: [Module]
allModules = [
  -- ... existing modules
  YourNewModule.module_,
  -- ... more modules
]
```

14.4. Language-Specific Generation

14.4.1. Haskell Code Generation

Generates native Haskell modules with proper imports and exports:

```
# Generate main Haskell modules
pixi run -e haskell stack-ghci
:| Hydra.Codegen
writeHaskell "src/gen-main/haskell" mainModules
```

```
# Generated structure
src/gen-main/haskell/
├── Hydra/
│   ├── Core.hs
│   ├── Graph.hs
│   └── Compute.hs
```

Generated Haskell features: * Type-safe data constructors * Pattern matching functions * Proper module hierarchies * Haddock documentation comments

14.4.2. Java Code Generation

Produces standard Java classes with proper packaging:

```
# Generate Java implementation
pixi run -e haskell generate-java

# Generated structure
hydra-java/src/gen-main/java/
├── hydra/
│   ├── core/
│   ├── graph/
│   └── compute/
```

Generated Java features: * Immutable data classes * Builder patterns for complex types * Proper package hierarchies * JavaDoc comments * Null-safety annotations

14.4.3. Python Code Generation (Future)

Planned Python generation will produce: * Type hints and annotations * Dataclasses for immutable types * Proper module structure * Sphinx documentation

14.4.4. Scala Code Generation (Future)

Planned Scala generation will produce: * Case classes for algebraic data types * Proper companion objects * Type-safe pattern matching * Scaladoc documentation

14.5. Customizing Code Generation

14.5.1. Adding New Language Support

To add support for a new target language:

1. Define Language Constraints:

```
-- In Hydra/Sources/Tier3/Ext/YourLanguage/Language.hs
yourLanguageConstraints :: LanguageConstraints
yourLanguageConstraints = LanguageConstraints {
  -- Define supported constructs
}
```

+ 2. Create Language Coder:

```
-- In Hydra/Ext/YourLanguage/Coder.hs
```

```
codeType :: Type -> Flow (Context) YourLanguageSyntax
codeTerm :: Term -> Flow (Context) YourLanguageSyntax
```

+ 3. Implement Serializer:

```
-- In Hydra/Ext/YourLanguage/Serializer.hs
serializeExpression :: YourLanguageSyntax -> String
```

+ 4. Register Generation Function:

```
writeYourLanguage :: FilePath -> [Module] -> IO ()
writeYourLanguage = generateSources yourLanguageConstraints codeType codeTerm
                      serializeExpression
```

14.5.2. Customizing Existing Generators

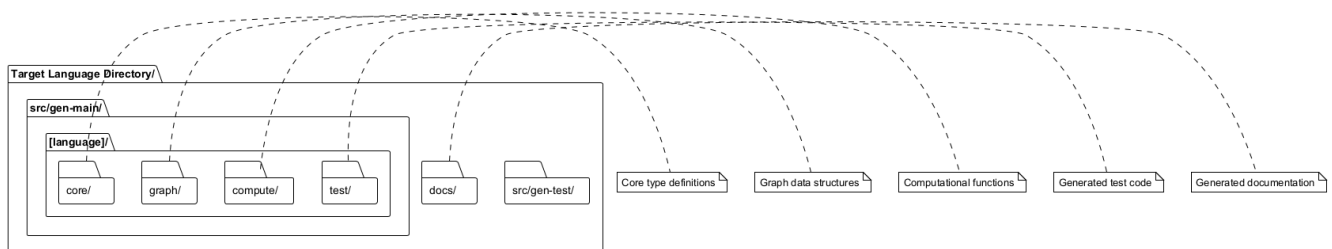
Modify generation behavior by updating:

- **Language constraints:** Change supported language features
- **Coders:** Modify how Hydra constructs map to target language
- **Serializers:** Change output formatting and style
- **Templates:** Modify file headers, imports, and boilerplate

14.6. Generated Code Organization

14.6.1. Directory Structure

Generated code follows consistent organization across languages:



14.6.2. Module Naming Conventions

- **Haskell:** `Hydra.Core`, `Hydra.Graph.Model`
- **Java:** `hydra.core`, `hydra.graph.model`
- **Python:** `hydra.core`, `hydra.graph.model`
- **Scala:** `hydra.core`, `hydra.graph.model`

14.6.3. Version Synchronization

Generated code includes version information:

```
-- Generated header includes version
-- This file was generated by Hydra 0.12.0
-- DO NOT EDIT: Changes will be overwritten
```

14.7. Development Workflow

14.7.1. Iterative Development

1. **Modify DSL sources** in `Hydra/Sources/`
2. **Regenerate code** with pixi tasks
3. **Build and test** in target languages
4. **Debug issues** by examining generated code
5. **Refine DSL sources** and repeat

```
# Development cycle
pixi run -e haskell generate-haskell
pixi run -e haskell stack-build
pixi run -e haskell stack-test

# If tests fail, examine generated code
pixi run -e haskell haskell-modules
pixi run -e haskell stack-ghci
-- :browse Hydra.YourModule
```

14.7.2. Debugging Generated Code

Common debugging approaches:

```
# Compare generated code before/after changes
git diff src/gen-main/

# Interactive exploration
pixi run -e haskell stack-ghci
:| Hydra.YourGeneratedModule
:browse Hydra.YourGeneratedModule
:type someGeneratedFunction

# Find specific functions
pixi run -e haskell haskell-functions | grep "yourFunction"
```

14.7.3. Testing Generated Code

```
# Test generated Haskell code
pixi run -e haskell stack-test

# Test specific modules
pixi run -e haskell stack-ghci
:| Test.Hydra.YourModuleTest
runTests

# Test generated Java code
pixi run -e java gradle test --tests "*YourModule*"
```

14.8. Code Generation Best Practices

14.8.1. DSL Design Guidelines

1. **Keep types simple:** Complex nested types are harder to generate cleanly
2. **Use consistent naming:** Names should translate well across languages
3. **Document thoroughly:** Comments in DSL become documentation in generated code
4. **Consider target languages:** Some constructs don't translate to all languages

14.8.2. Generation Quality

1. **Always regenerate after changes:** Don't edit generated code directly
2. **Test all target languages:** Ensure generation works for all supported languages
3. **Verify compilation:** Generated code must compile without warnings
4. **Check performance:** Generated code should be reasonably efficient

14.8.3. Version Control

1. **Commit generated code:** Include generated code in version control
2. **Review generated changes:** Always review diffs in generated code
3. **Coordinate updates:** Update all language implementations together
4. **Tag releases:** Ensure generated code matches tagged versions

14.9. Advanced Topics

14.9.1. Custom Code Templates

For specialized generation needs, create custom templates:

```
-- Custom template for specific language features
```

```
customTemplate :: TemplateConfig -> String
customTemplate config = [str|
  // Generated by Hydra #{version}
  // Custom template for #{languageName}

  #{generatedContent}
|]
```

14.9.2. Conditional Generation

Generate different code based on target language capabilities:

```
generateConditionally :: LanguageConstraints -> Type -> Flow s Expression
generateConditionally constraints typ
  | supportsGenerics constraints = generateGenericVersion typ
  | otherwise = generateNonGenericVersion typ
```

14.9.3. Cross-Language Testing

Ensure behavioral parity across generated implementations:

```
# Generate test data in one language
pixi run -e haskell run-example-graphson

# Verify same results in other languages
pixi run -e java gradle run --args="verify-graphson"
```

14.10. Troubleshooting

14.10.1. Common Issues

Generation Fails

Symptoms: Code generation throws errors or produces invalid code.

Solutions: 1. Check DSL syntax in source definitions 2. Verify all imports are available 3. Ensure proper tier dependencies 4. Review language constraints compatibility

```
# Debug generation issues
pixi run -e haskell stack-ghci
:set -XOverloadedStrings
:l Hydra.Codegen
-- Try individual generation functions
```

Generated Code Won't Compile

Symptoms: Generated code has syntax errors or missing imports.

Solutions: 1. Check coder implementation for target language 2. Verify serializer produces valid syntax 3. Update language constraints if needed 4. Review import generation logic

Performance Issues

Symptoms: Generated code is slower than expected.

Solutions: 1. Profile generated code execution 2. Optimize coder algorithms 3. Use language-specific optimizations 4. Consider hand-written alternatives for hot paths

14.10.2. Diagnostic Commands

```
# Check what modules are available for generation
pixi run -e haskell haskell-modules

# Find specific functions in generated code
pixi run -e haskell haskell-functions | grep -i "search-term"

# Interactive debugging
pixi run -e haskell stack-ghci
:l Hydra.Codegen
:t writeHaskell
:t writeJava

# Verify generation output
ls -la src/gen-main/haskell/Hydra/
ls -la ../hydra-java/src/gen-main/java/hydra/
```

14.11. Related Documentation

- [Developer Guide](#) - Overall development process
- [Dependency Tiers](#) - Understanding code organization
- [Pixi Haskell Tasks](#) - Generation commands and workflows
- [Implementor Guide](#) - Creating new language implementations

Chapter 15. Part V: Documentation System

Chapter 16. Documentation System

This section describes Hydra's documentation system, including the build process, authoring guidelines, and maintenance workflows.

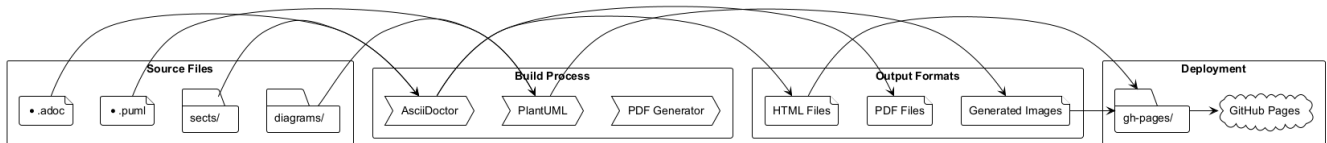
16.1. Overview

Hydra uses a modern documentation system built on AsciiDoc with PlantUML diagram integration. The system is designed to generate both HTML and PDF outputs, with automated deployment to GitHub Pages for online access.

16.1.1. Key Features

- **AsciiDoc Format:** Semantic markup for technical documentation
- **PlantUML Diagrams:** Professional diagrams generated from source code
- **Multi-format Output:** HTML for web, PDF for distribution
- **Pixi Integration:** Automated builds using pixi task system
- **GitHub Pages:** Automated deployment and hosting
- **Modular Structure:** Reusable sections and includes

16.1.2. Documentation Architecture



16.2. Build System

16.2.1. Pixi Tasks for Documentation

The documentation system integrates with Hydra's pixi-based build system through dedicated tasks:

Quick Build Tasks

```
# Build all HTML documentation
pixi run docs-html

# Build all PDF documentation
pixi run docs-pdf

# Build both HTML and PDF
pixi run docs-all
```

```
# Serve documentation locally
pixi run docs-serve
```

Individual Guide Tasks

```
# Build specific guides (HTML)
pixi run -e docs docs-build-developer-guide
pixi run -e docs docs-build-implementor-guide
pixi run -e docs docs-build-operator-guide

# Build specific guides (PDF)
pixi run -e docs docs-build-pdf-developer
pixi run -e docs docs-build-pdf-implementor
pixi run -e docs docs-build-pdf-operator
```

Maintenance Tasks

```
# Clean output directory
pixi run -e docs docs-clean

# Regenerate all diagrams
pixi run -e docs docs-generate-diagrams

# Prepare build environment
pixi run -e docs docs-prepare
```

16.2.2. Build Dependencies

The documentation environment includes:

- **Ruby:** Runtime for AsciiDoctor tools
- **asciidoctor:** Main AsciiDoc processor
- **rb-asciidoctor-pdf:** PDF generation support
- **rb-asciidoctor-diagram:** PlantUML integration
- **plantuml:** Diagram generation engine
- **graphviz:** Graph layout support

16.2.3. Build Process Flow

1. Preparation Phase

- Clean output directory (**docs-output/**)
- Generate PlantUML diagrams from source files
- Copy static assets (images, CSS)

2. Processing Phase

- Process AsciiDoc files with diagram extension
- Resolve includes and cross-references
- Apply syntax highlighting and formatting

3. Output Phase

- Generate HTML files with embedded diagrams
- Create PDF versions using asciidoctor-pdf
- Copy all assets to output directory

4. Deployment Phase (optional)

- Prepare files for GitHub Pages
- Create deployment-ready directory structure
- Generate navigation and index files

16.3. Authoring Documentation

16.3.1. File Organization

The documentation follows a structured organization:

```
docs/
├── index.adoc           # Main index
├── developer-guide.adoc # Complete developer guide
├── implementor-guide.adoc # Implementor guide
├── operator-guide.adoc  # Operator guide
└── sects/               # Individual sections
    ├── architecture-setup.adoc
    ├── build-system.adoc
    ├── pixi-setup.adoc
    ├── documentation-system.adoc # This file
    └── ... (other sections)
```

16.3.2. AsciiDoc Conventions

Document Headers

All documentation files should include consistent headers:

```
= Document Title
:toc: left
:toclevels: 3
:sectlinks:
:sectanchors:
:source-highlighter: pygments
```

Introduction paragraph explaining the document's purpose.

Section Hierarchy

Use consistent section hierarchy:

```
= Document Title      (Level 0 - Document title)
== Main Section       (Level 1 - Main sections)
=== Subsection        (Level 2 - Subsections)
==== Sub-subsection   (Level 3 - Detailed topics)
===== Details       (Level 4 - Specific details)
```

Code Examples

Format code examples with proper syntax highlighting:

```
[source,bash]
```

```
pixi run docs-html
```

```
[source,haskell]
```

```
module Main where main = putStrLn "Hello, Hydra!"
```

Cross-References

Use semantic cross-references between documents:

```
See the <<build-system#,Build System Guide>> for details.
Refer to <<pixi-setup#troubleshooting,Troubleshooting Section>>.
```

Notes and Admonitions

Use admonitions to highlight important information:

```
NOTE: This provides additional context.
```

```
TIP: This offers helpful advice.
```

```
WARNING: This warns about potential issues.
```

IMPORTANT: This highlights critical information.

16.3.3. PlantUML Diagrams

Creating Diagrams

1. **Create source file** in `docs/diagrams/` directory
2. **Follow naming convention:** `descriptive-name.puml`
3. **Use consistent styling:**

```
@startuml
!theme plain
skinparam backgroundColor white
skinparam defaultFontName Arial

' Your diagram content here

@enduml
```

Embedding Diagrams

Include diagrams in documentation using:

```
[plantuml,diaqram-name,png]
....
@startuml
!theme plain
skinparam backgroundColor white
skinparam defaultFontName Arial

rectangle "Source Files" {
    file "*.adoc" as adoc
    file "*.puml" as puml
    folder "sects/" as sections
    folder "diagrams/" as diagrams
}

rectangle "Build Process" {
    process "AsciiDoctor" as ad
    process "PlantUML" as plant
    process "PDF Generator" as pdf
}

rectangle "Output Formats" {
    file "HTML Files" as html
    file "PDF Files" as pdfs
    file "Generated Images" as images
}
```

```

}

rectangle "Deployment" {
  cloud "GitHub Pages" as gh
  folder "gh-pages/" as ghdir
}

adoc -> ad
puml -> plant
sections -> ad
diagrams -> plant

ad -> html
ad -> pdfs
plant -> images

html -> ghdir
images -> ghdir
ghdir -> gh

@enduml
....

```

Diagram Types

Common diagram types used in Hydra documentation:

- **Sequence Diagrams:** For process flows and interactions
- **Component Diagrams:** For system architecture
- **Activity Diagrams:** For workflows and procedures
- **Class Diagrams:** For type hierarchies
- **Deployment Diagrams:** For system deployment

16.4. GitHub Pages Integration

16.4.1. Automated Deployment

The documentation system supports automated GitHub Pages deployment:

```

# Build and prepare for GitHub Pages
pixi run gh-pages-build

```

This creates a **gh-pages/** directory with:

- All HTML documentation files
- Generated diagram images

- Navigation and index files
- Required GitHub Pages configuration

16.4.2. Deployment Process

1. **Build documentation** using pixi tasks
2. **Run GitHub Pages build** to prepare files
3. **Commit and push** to gh-pages branch:

```
cd gh-pages
git add .
git commit -m "Update documentation"
git push origin gh-pages
```

1. **Verify deployment** at GitHub Pages URL

16.4.3. Custom Domain Support

To use a custom domain:

1. **Add CNAME file** in gh-pages directory
2. **Configure DNS** for your domain
3. **Update repository settings** in GitHub

16.5. Development Workflow

16.5.1. Daily Documentation Work

1. **Edit content** in `.adoc` files
2. **Create/modify diagrams** in `diagrams/` directory
3. **Build and preview locally:** [source,bash] ---- pixi run docs-html pixi run docs-serve # Visit <http://localhost:8080> ----
4. **Iterate** until satisfied with results

16.5.2. Before Committing Changes

1. **Build all formats** to check for errors: [source,bash] ---- pixi run docs-all ----
2. **Validate output** in `docs-output/` directory
3. **Test GitHub Pages build:** [source,bash] ---- pixi run gh-pages-build ----
4. **Review generated files** for correctness

16.5.3. Release Documentation

1. **Update version references** throughout documentation
2. **Build complete documentation set**
3. **Deploy to GitHub Pages**
4. **Update any external references**

16.6. Maintenance and Best Practices

16.6.1. Content Guidelines

- **Keep content current** with software changes
- **Use clear, concise language**
- **Include practical examples** for all concepts
- **Maintain consistent terminology** across documents
- **Test all code examples** before including them

16.6.2. Diagram Guidelines

- **Keep diagrams simple** and focused
- **Use consistent styling** across all diagrams
- **Include descriptive labels** and notes
- **Version control source files** (`.puml` files)
- **Optimize for readability** at normal viewing sizes

16.6.3. Quality Assurance

- **Regular content review** for accuracy
- **Link validation** to prevent broken references
- **Build testing** across all output formats
- **User feedback integration** from community
- **Version synchronization** with codebase changes

16.6.4. Performance Considerations

- **Diagram caching** to speed up builds
- **Incremental builds** when possible
- **Output optimization** for web delivery
- **Image compression** for faster loading
- **Build parallelization** for complex documents

16.7. Troubleshooting

16.7.1. Common Issues

PlantUML Rendering Problems

- **Java requirement:** Ensure Java runtime is available
- **Syntax validation:** Check `.puml` file syntax
- **Path resolution:** Verify include paths are correct
- **Memory limits:** Increase Java heap for complex diagrams

AsciiDoc Processing Errors

- **Include failures:** Check file paths and permissions
- **Cross-reference errors:** Validate anchor names and targets
- **Attribute problems:** Ensure document attributes are set
- **Encoding issues:** Use UTF-8 encoding consistently

Build Environment Issues

- **Dependency problems:** Verify all gems are installed
- **Version conflicts:** Check Ruby and gem versions
- **Path configuration:** Ensure tools are in system PATH
- **Permission errors:** Check file and directory permissions

16.7.2. Debugging Techniques

```
# Test individual components
plantuml docs/diagrams/test-diagram.puml
asciidoctor docs/test-document.adoc

# Verbose build output
asciidoctor -v -r asciidoctor-diagram docs/developer-guide.adoc

# Check environment
pixi list -e docs
which asciidoctor
which plantuml
```

16.8. Future Enhancements

16.8.1. Planned Improvements

- **Interactive diagrams** using modern web technologies
- **Multi-language support** for international documentation
- **API documentation integration** from source code
- **Advanced search capabilities** for large documentation sets
- **Collaborative editing** workflows for distributed teams

16.8.2. Extension Opportunities

- **Custom AsciiDoc extensions** for Hydra-specific markup
- **Automated diagram generation** from code structures
- **Integration testing** for documentation examples
- **Metrics and analytics** for documentation usage
- **Mobile-optimized rendering** for improved accessibility

16.9. Contributing to Documentation

16.9.1. Getting Started

1. **Set up environment:** [source,bash] ---- pixi install -e docs ----
2. **Learn AsciiDoc syntax** from official documentation
3. **Study existing documentation** for style and patterns
4. **Start with small contributions** like typo fixes
5. **Progress to larger sections** as you gain familiarity

16.9.2. Contribution Process

1. **Create feature branch** for your changes
2. **Follow style guidelines** established in this document
3. **Test your changes** thoroughly
4. **Submit pull request** with clear description
5. **Address review feedback** promptly

16.9.3. Review Criteria

- **Technical accuracy:** All content must be correct
- **Style consistency:** Follow established patterns
- **Completeness:** Include all necessary information
- **Clarity:** Write for the intended audience

- **Testing:** Verify all examples work correctly

For questions about the documentation system, consult the [docs/README.md](#) file or reach out via the project's Discord community.

Chapter 17. Part VI: Release Management

Chapter 18. Release Management

This document covers the complete release management process for all Hydra components, including version coordination, testing procedures, and deployment workflows.

18.1. Release Overview

Hydra follows a coordinated multi-component release strategy where all language implementations maintain version parity. The primary release process is driven by Hydra-Haskell, with other implementations following in a coordinated fashion.

18.1.1. Release Components

- **hydra-haskell**: Complete Hydra implementation and source of truth
- **hydra-java**: Production-ready Java implementation
- **hydra-python**: Python implementation (in development)
- **hydra-scala**: Scala implementation (planned)
- **hydra-ext**: Extensions and demonstration code

18.1.2. Version Strategy

All components use semantic versioning (MAJOR.MINOR.PATCH) and maintain version parity:

- **Major versions**: Breaking changes to the Hydra kernel or API
- **Minor versions**: New features, kernel extensions, language additions
- **Patch versions**: Bug fixes, documentation updates, minor improvements

18.2. Pre-Release Process

18.2.1. Version Planning

```
# Check current versions across components
grep -r "version.*=" pixi.toml
grep -r "version" hydra-*/package.yaml
grep -r "version" hydra-*/build.gradle
```

18.2.2. Pre-Release Readiness Checks

1. Project Updates

- Review and close relevant GitHub issues
- Update documentation for new features
- Ensure all planned features are complete

- Review security considerations

2. Dependency Management

```
# Update Haskell dependencies
cd hydra-haskell
stack list-dependencies

# Check for outdated dependencies
stack outdated

# Update Java dependencies
cd ../hydra-java
./gradlew dependencyUpdates
```

3. Stack LTS Upgrade (Haskell)

Update Stack resolver in `hydra-haskell/stack.yaml`:

```
resolver: lts-22.0 # Update to latest LTS
```

Then rebuild and test:

```
pixi run -e haskell stack-clean
pixi run -e haskell stack-build
pixi run -e haskell stack-test
```

4. Cross-Implementation Testing

```
# Test all implementations
pixi run build-all
pixi run -e haskell stack-test
pixi run -e java gradle test

# Verify generated code parity
pixi run -e haskell generate-haskell
pixi run -e haskell generate-java

# Test code generation examples
pixi run -e haskell run-example-graphson
pixi run -e haskell check-copilot-data
```

18.3. Hydra-Haskell Release Process

18.3.1. Version Preparation

1. Update Version Numbers

Update version in `hydra-haskell/package.yaml`:

```
version: '0.12.1'
```

Update corresponding versions: `* hydra-java/build.gradle * hydra-ext/package.yaml * pixi.toml`

2. Update Changelog

Add release notes to `CHANGELOG.md`:

```
## [0.12.1] - 2024-01-15

### Added
- New primitive functions for graph traversal
- Enhanced JSON schema support

### Changed
- Improved error messages in type checker
- Updated dependency versions

### Fixed
- Memory leak in large graph processing
- Edge case in code generation

### Breaking Changes
- Renamed 'oldFunction' to 'newFunction'
```

3. Commit Version Changes

```
git add .
git commit -m "Bump Hydra version to 0.12.1"
git push origin main
```

18.3.2. Release to Hackage

1. Create Release Package

```
cd hydra-haskell
pixi run -e haskell stack sdist
```

This creates the package in `.stack-work/dist/x86_64-linux/Cabal-/hydra-haskell-.tar.gz`

2. Upload to Hackage

1. Log in to <https://hackage.haskell.org>
2. Navigate to "Upload package"
3. Upload the generated `.tar.gz` file
4. Review the package contents and metadata
5. Publish the package

3. Monitor Release Status

- Check build logs on Hackage
- Monitor for any reported issues
- Verify package appears in search results
- Test installation: `cabal install hydra-haskell`

18.3.3. Tag the Release

Once Hackage release is confirmed successful:

```
# Create and push the release tag
git tag 0.12.1 -m "0.12.1 release" [commit-hash]
git push --tags

# Verify tag was created
git tag -l "0.12.*"
```

18.4. Hydra-Java Release Process

Java releases are typically coordinated with Haskell releases but can be independent for Java-specific fixes.

18.4.1. Pre-Release Steps

1. Sync with Haskell Version

Ensure Java implementation includes latest generated code:

```
# Generate latest Java code
pixi run -e haskell generate-java

# Commit any generated code updates
git add hydra-java/src/gen-main/java/
git commit -m "Update generated Java code for 0.12.1"
```

2. Update Java Version

Update version in `hydra-java/build.gradle`:

```
version = '0.12.1'
```

3. Update JavaDocs

```
# Regenerate JavaDocs
rm -rf hydra-java/javadoc
cd hydra-java
./bin/update-javadoc.sh

# Commit updated documentation
git add javadoc/
git commit -m "Update JavaDocs for 0.12.1 release"
git push origin main
```

4. Update Documentation Branch

```
# Switch to docs branch and sync
git checkout docs
git pull origin main
git push origin docs
```

This triggers GitHub Pages generation of online JavaDocs.

18.4.2. Release to Maven Central

1. Build and Stage

```
cd hydra-java
pixi run -e java gradle build
./gradlew publishToSonatype
```

2. Release via Sonatype

1. Log in to <https://oss.sonatype.org>
2. Navigate to "Staging Repositories"
3. Find your staging repository (com.fortytwo.hydra-*)
4. Click "Close" and provide message: "Hydra-Java 0.12.1 release"
5. Wait for validation (several minutes)
6. Click "Release" when available

7. Provide release message: "Hydra 0.12.1 release"

3. Verify Release

```
# Check Maven Central (may take up to 2 hours)
# https://search.maven.org/search?q=g:net.fortytwo.hydra

# Test installation
gradle init
# Add dependency and test
```

18.5. Multi-Component Release Coordination

18.5.1. Version Synchronization

```
# Script to update all versions consistently
#!/bin/bash
NEW_VERSION="0.12.1"

# Update Haskell
sed -i "s/version: '.*'/version: '$NEW_VERSION'/" hydra-haskell/package.yaml

# Update Java
sed -i "s/version = '.*'/version = '$NEW_VERSION'/" hydra-java/build.gradle

# Update Extensions
sed -i "s/version: '.*'/version: '$NEW_VERSION'/" hydra-ext/package.yaml

# Update pixi.toml
sed -i "s/version = \".*\"/version = \"$NEW_VERSION\"/" pixi.toml
```

18.5.2. Testing Cross-Implementation Compatibility

```
# Comprehensive compatibility test
pixi run validate
pixi run build-all
pixi run -e haskell stack-test
pixi run -e java gradle test

# Test generated code consistency
pixi run -e haskell generate-haskell
pixi run -e haskell generate-java

# Verify no unexpected changes
git diff --name-only
```

18.6. Hotfix Release Process

For urgent bug fixes that can't wait for the next regular release:

18.6.1. 1. Create Hotfix Branch

```
git checkout main
git pull origin main
git checkout -b hotfix/0.12.2
```

18.6.2. 2. Implement and Test Fix

```
# Make necessary changes
# ...

# Test thoroughly
pixi run -e haskell stack-test
pixi run -e java gradle test
pixi run build-all
```

18.6.3. 3. Update Version (Patch Only)

```
# Update to patch version (0.12.1 -> 0.12.2)
# Update all version files
git commit -m "Bump version to 0.12.2 for hotfix"
```

18.6.4. 4. Merge and Release

```
git checkout main
git merge hotfix/0.12.2
git push origin main

# Follow normal release process for this patch version
# Tag and release to Hackage/Maven Central
```

18.7. Post-Release Tasks

18.7.1. 1. Update GitHub Release

1. Go to <https://github.com/CategoricalData/hydra/releases>
2. Click "Draft a new release"
3. Select the version tag (e.g., 0.12.1)

4. Fill in release notes from CHANGELOG.md
5. Upload any release artifacts
6. Publish the release

18.7.2. 2. Update Documentation

```
# Update version references in documentation
grep -r "0\..12\..0" docs/
# Update to new version as needed
```

18.7.3. 3. Notify Community

- Post release announcement on Discord (#releases channel)
- Update any external documentation
- Notify downstream users of breaking changes

18.7.4. 4. Monitor for Issues

- Watch for bug reports in GitHub issues
- Monitor Discord for user questions
- Check build status on various platforms

18.8. Release Quality Checklist

Before any release, verify:

- ☐ All tests pass on CI/CD
- ☐ Documentation is up to date
- ☐ Version numbers are consistent across all components
- ☐ Generated code is current and committed
- ☐ CHANGELOG.md includes all changes
- ☐ No known critical bugs
- ☐ Performance regressions addressed
- ☐ Breaking changes are documented
- ☐ Migration guides provided (if needed)
- ☐ Cross-platform compatibility verified

18.9. Emergency Procedures

18.9.1. Reverting a Release

If a critical issue is found immediately after release:

```
# For Hackage (contact maintainers)
# Email: admin@hackage.haskell.org

# For Maven Central (within 24 hours)
# Contact Sonatype support

# For GitHub
git tag -d 0.12.1
git push origin :refs/tags/0.12.1
```

18.9.2. Hotfix Communication

Template for urgent hotfix announcements:

[Alert] Hydra Hotfix Release 0.12.2

Critical bug fix for issue #XXX affecting [component].

Affected versions: 0.12.0 - 0.12.1 **Fixed in:** 0.12.2 **Severity:** High/Medium/Low

Action required: Update to 0.12.2 immediately **Update command:** - Haskell: `stack update && stack install hydra-haskell` - Java: Update version in build.gradle to 0.12.2

Details: [link to issue/PR]

18.10. Automation and Tools

18.10.1. Release Scripts

Consider creating release automation scripts:

```
# scripts/prepare-release.sh
#!/bin/bash
set -e

VERSION=$1
if [ -z "$VERSION" ]; then
    echo "Usage: $0 <version>"
    exit 1
fi

echo "Preparing release $VERSION..."

# Update versions
# Run tests
```

```
# Generate code
# Commit changes
# Create PR
```

18.10.2. CI/CD Integration

Ensure your CI/CD pipeline includes:

- Automated version validation
- Cross-platform testing
- Generated code verification
- Documentation building
- Release artifact creation

18.11. Related Documentation

- [Developer Guide](#) - Overall development process
- [Pixi Haskell Tasks](#) - Build and test commands
- Main repository README for contributor guidelines
- Individual component documentation for implementation-specific details

Chapter 19. Appendices

Appendix A: Development Resources

- Main Repository: <https://github.com/CategoricalData/hydra>
- LambdaGraph Discord: <https://bit.ly/lg-discord>
- Hackage Package: <https://hackage.haskell.org/package/hydra-haskell>
- Maven Central: <https://search.maven.org/search?q=net.fortytwo.hydra>
- JavaDocs: Available via GitHub pages

Appendix B: Common Development Tasks

19.B.1. Setting Up Development Environment

```
# Clone the repository
git clone https://github.com/CategoricalData/hydra
cd hydra

# Install all build dependencies
pixi install

# Validate the setup
pixi run validate

# Setup specific environments
pixi run -e haskell haskell-setup # Complete Haskell development
pixi install -e java              # Java development environment
pixi install -e python            # Python development environment
pixi install -e scala             # Scala development environment

# Build all components
pixi run build-all
```

Appendix C: Complete Pixi Tasks Reference

This appendix provides a comprehensive reference to all available pixi tasks for Hydra development.

19.C.1. General Build and Setup Tasks

```
# Core build tasks
pixi run build-all      # Build all components
pixi run build-java      # Build Java components
pixi run build-python    # Build Python components
```

```
pixi run build-scala      # Build Scala components
pixi run build-haskell    # Build Haskell components
pixi run build-ext        # Build extensions
pixi run clean            # Clean build artifacts
pixi run validate         # Validate setup
```

19.C.2. Java Development Tasks

```
# Java environment tasks
pixi run -e java gradle      # Run Gradle wrapper
pixi run -e java gradle-tasks # List available Gradle tasks
pixi run -e java gradle-build # Build Java components
pixi run -e java gradle-clean # Clean Java build artifacts
```

19.C.3. Python Development Tasks

```
# Python environment tasks
pixi run build-python      # Build Python components
pixi run -e python python --version
pixi run -e python pip list
```

19.C.4. Scala Development Tasks

```
# Scala environment tasks
pixi run build-scala      # Build Scala components
pixi run -e scala sbt version
pixi run -e scala sbt compile
pixi run -e scala sbt test
```

19.C.5. Haskell Development Tasks

Setup and Build

```
pixi run -e haskell haskell-setup      # Complete setup
pixi run -e haskell haskell-setup-all  # Setup all packages
pixi run -e haskell stack-setup        # Stack setup
pixi run -e haskell stack-build        # Build all packages
pixi run -e haskell haskell-build-all # Build all Haskell packages
pixi run -e haskell hydra-build        # Build core package
```

Testing

```
pixi run -e haskell stack-test      # Test all packages
```

```
pixi run -e haskell haskell-test-all # Test all Haskell packages
pixi run -e haskell hydra-test        # Test core package
pixi run -e haskell ext-test          # Test extensions
```

Interactive Development

```
pixi run -e haskell stack-ghci      # Multi-package GHCi
pixi run -e haskell ghci-all        # All packages GHCi
pixi run -e haskell hydra-ghci      # Core package GHCi
pixi run -e haskell ext-ghci        # Extensions GHCi
pixi run -e haskell ghci-interactive # Interactive GHCi
pixi run -e haskell ghci-all-fixed  # Fixed GHCi variant
pixi run -e haskell ghci-system      # System GHCi
pixi run -e haskell ghci-minimal     # Minimal conda interference
```

Code Generation

```
pixi run -e haskell generate-java    # Generate Java code
pixi run -e haskell generate-haskell # Generate Haskell code
```

Data and Testing

```
pixi run -e haskell check-data      # Check source data
pixi run -e haskell check-graphson  # Check GraphSON files
pixi run -e haskell check-copilot-data # Check copilot data
pixi run -e haskell check-sales-data # Check sales data
pixi run -e haskell run-copilot-graphson # Generate copilot GraphSON
pixi run -e haskell run-example-graphson # Generate example GraphSON
pixi run -e haskell graphson-help    # GraphSON function help
```

Exploration and Debugging

```
pixi run -e haskell haskell-modules # List modules
pixi run -e haskell haskell-functions # Find functions
```

Cleanup

```
pixi run -e haskell stack-clean      # Clean Stack builds
pixi run -e haskell haskell-clean-all # Clean all Haskell builds
pixi run -e haskell clear-stack-cache-haskell # Clear core cache
pixi run -e haskell clear-stack-cache-ext # Clear extensions cache
pixi run -e haskell clear-stack-cache-all # Clear all caches
```

19.C.6. Development Workflow Recommendations

1. **Initial Setup:** `pixi run validate` → `pixi run -e haskell haskell-setup` → `pixi run build-all`
2. **Daily Development:** `pixi run -e haskell stack-ghci` for interactive development
3. **Testing Changes:** `pixi run -e haskell stack-test` → `pixi run build-all`
4. **Code Generation:** `pixi run -e haskell generate-haskell` → `pixi run -e haskell generate-java`
5. **Troubleshooting:** `pixi run clean` → `pixi run -e haskell clear-stack-cache-all` → rebuild

19.C.7. Debugging Build Issues

1. Check dependency tiers - ensure no circular dependencies
2. Verify all primitive functions are implemented
3. Check that generated code is up to date
4. Review error logs for missing imports or type mismatches

```
# Clean and rebuild when issues arise
pixi run clean
pixi run build-all

# For Haskell-specific issues
pixi run -e haskell stack-clean
pixi run -e haskell clear-stack-cache-all
pixi run -e haskell stack-build

# For Java-specific issues
pixi run -e java gradle-clean
pixi run -e java gradle-build
```

19.C.8. Contributing Guidelines

1. Follow the established dependency tier structure
2. Write comprehensive tests for new features
3. Update documentation for any API changes
4. Ensure all language implementations maintain parity
5. Follow the release process for version bumps

Colophon

This developer guide covers the internal architecture and development processes for Hydra core development. It is maintained alongside the codebase and should be updated whenever significant architectural changes are made.

For the latest version of this guide and related documentation, see the [docs/](#) directory in the Hydra repository.